

Sleep-Time Compute를 위한 Training Background

학습을 처음 접하는 시스템·메모리 연구자를 위한 독립 참고서

이 책은 “용어 사전”이 아니라, 데이터가 어떻게 손실과 기울기로 바뀌고, 그 기울기가 어떤 상태를 얼마나 오래 바꾸며, 그 결과를 어떻게 검증·출판·롤백하는지 하나의 시스템으로 설명한다. Study Paper의 [배경: concept-id] 링크는 이 책의 해당 개념으로 연결된다.

읽는 방법

각 장은 직관에서 시작해, 필요한 만큼의 최소 수식, 작은수치 예, Sleep-time compute와의 연결, 마지막으로 시스템 비용 순서로 진행한다. 미적분을 몰라도 “기울기는 어느 쪽으로 움직이면 손실이 줄어드는지를 알려 주는 경사”라는 직관에서 출발할 수 있다. 수식은 정확한 상태와 비용을 구분하기 위해 사용하며, 독자가 계산을 암기하는 것이 목적은 아니다.

핵심 이 책이 답하는 질문

1. 모델이 “배운다”는 것은 어떤 수치 상태가 바뀐다는 뜻인가?
2. full fine-tuning, LoRA, distillation, replay, RL 은 데이터 · 목표 · 상태가 어떻게 다른가?
3. 계속 배우면 왜 잊고, 용량은 어디서 소진되며, 무엇을 검증해야 하는가?
4. wake 학습과 sleep 학습을 분리하면 어떤 compute, memory, I/O, rollback 비용이 생기는가?

AI assistance disclosure. During the preparation of this work the authors used Codex GPT-5.6 for evidence synthesis, LaTeX drafting, and artifact QA. All content was reviewed and edited by the authors, who take full responsibility for the accuracy and integrity of the work.

Contents

읽는 방법	i
1 학습은 상태를 바꾸는 검증 가능한 절차다	1
1.1 추론과 학습을 먼저 분리하자	1
1.2 목적과 손실: 무엇을 좋아지게 할 것인가	1
1.2.1 스칼라 예: 한 숫자를 맞추는 모델	1
1.3 최적화 목표와 제품 목표는 다르다	2
1.4 Sleep-time compute 와의 연결	2
2 순전파, 역전파, optimizer 를 한 흐름으로 보기	4
2.1 순전파는 값을 만들고, 역전파는 책임을 나눈다	4
2.2 Optimizer 는 update rule 과 기억 상태를 가진다	4
2.3 Activation memory 와 checkpointing	5
2.4 Gradient accumulation 과 effective batch	5
2.5 Sleep-time operator 로 번역하기	6
3 배치, 샘플링, 검증: 데이터 흐름이 결과를 결정한다	7
3.1 Batch 는 단지 GPU 를 채우는 단위가 아니다	7
3.2 Sampling 은 무엇을 기억하고 무엇을 잊을지 정한다	7
3.3 Training, validation, test 의 역할	7
3.4 Overfitting 과 generalization	8
3.5 통계적 유의성과 운영적 유의성	8
4 배포 전 · 후의 학습 regime 지도	9
4.1 Regime 를 네 질문으로 구분한다	9
4.2 Pre-training 과 continued pre-training	9
4.3 Full fine-tuning 과 instruction tuning	10
4.4 Preference optimization	10
4.5 Sleep-time 은 새로운 objective 가 아니라 lifecycle 축이다	11
5 지도 신호, 자기지도, representation 과 언어모델 loss	12
5.1 정답은 어디서 오는가	12
5.2 Representation 은 저장소와 다르다	12
5.3 Logit, softmax, cross-entropy	12
5.4 Mask 와 sequence-level objective	13
5.5 Systems 관점: data transformation 도 compute 다	13

6 Distillation: 큰 경험을 작은 상태로 옮기기	14
6.1 교사와 학생	14
6.2 Memory consolidation 으로서 distillation	14
6.3 무엇이 압축 과정에서 사라지는가	15
6.4 Recursive distillation 과 model collapse	15
7 PEFT, LoRA, adapter, MoE: 얼마나 작은 상태를 바꿀 것인가	16
7.1 Trainable state 를 줄이는 이유	16
7.2 LoRA 의 최소 수식	16
7.3 Adapter module 과 routing	16
7.4 Mixture of Experts 는 PEFT 와 다른 축이다	17
7.5 Selective promotion 이라는 hybrid	17
8 Sleep 이 먹는 것: data curation, augmentation, replay	19
8.1 Log 와 training example 은 다르다	19
8.2 Augmentation 은 의미를 보존한다는 가정이다	20
8.3 Replay: 과거를 다시 보여 주는 가장 직접적인 기준선	20
8.4 평가 누수와 시간 누수	21
9 계속 배우면서 잊지 않기: continual learning 의 문제 설정	22
9.1 왜 새 학습이 과거 능력을 지우는가	22
9.2 성능 행렬로 retention 과 transfer 를 분리한다	22
9.3 네 가족의 방법과 숨은 비용	23
9.4 온라인 평가와 shadow publication	23
10 보존 · 격리 · 확장 · 압축: 안정성-가소성 mechanism	25
10.1 Regularization: 중요한 parameter 는 덜 움직인다	25
10.2 Gradient constraint: 과거 손실을 늘리는 방향을 피한다	25
10.3 Isolation 과 growth: 충돌할 parameter 를 나눈다	25
10.4 Capacity 는 bytes 만이 아니라 interference 와 운영 한도다	26
10.5 Admission, eviction, consolidation 은 하나의 정책이다	26
11 강화학습의 최소 토대: sleep 을 하나의 의사결정 문제로 보기	28
11.1 State, action, reward, trajectory	28
11.2 Return, policy, value	28
11.3 Policy gradient 와 update 가 실제로 의미하는 것	29
11.4 On-policy 와 off-policy 의 systems 차이	29

12 Preference learning 에서 memory policy 까지	30
12.1 Preference pair 와 reward model	30
12.2 DPO: RL rollout 없이 preference loss 를 쓰는 선택지	30
12.3 Memory controller 가 고르는 여섯 종류의 행동	30
12.4 Offline evaluation 이 어려운 이유	31
13 Meta-learning, fast weights, test-time training	33
13.1 Learning to learn: 초기값과 update rule 을 학습한다	33
13.2 Fast weights: 상태의 수명을 설계한다	33
13.3 Test-time training 의 정확한 경계	33
13.4 Reset, accumulation, leakage	34
14 Model editing 과 unlearning: 좁게 바꾸고 정확히 되돌리기	35
14.1 Model editing 의 네 평가축	35
14.2 많은 edit 는 다시 continual learning 과 capacity 문제가 된다	35
14.3 Unlearning 은 delete key 보다 강한 요구다	36
14.4 Provenance graph 가 architecture choice 를 바꾼다	36
15 Training systems: HBM, 통신, checkpoint, publication	37
15.1 한 step 의 memory accounting	37
15.2 병렬화는 무엇을 복제하고 무엇을 통신하는가	37
15.3 Checkpoint 가 아니라 release bundle 을 version 한다	39
15.4 Memory device 관점의 기회	39
16 End-to-end mapping: wake 경험에서 검증된 장기기억까지	40
16.1 두 기억 destination 과 hybrid promotion	40
16.2 모든 sleep operator 를 같은 schema 로 기록한다	40
16.3 Worked example: 반복되는 사용자 correction	41
16.4 Break-even 을 묻는 최소 식	41
16.5 운영 decision tree	41
용어 사전과 개념 의존성	43
참고문헌	49

List of Figures

1.1	데이터에서 출판까지 이어지는 학습 상태 전환. Sleep-time compute 는 이파이프라인의 일부를 응답 latency 경로 밖으로 옮기지만 검증 단계를 없애지 않는다.	3
4.1	배포 시점과 변경 상태 크기로 본 training regime. Test-time training 은 wake-path, sleep-time training 은 deferred path 에 놓이지만 둘 다 외부 기억과 parametric 상태를 선택할 수 있다.	9
9.1	Continual-learning 방법을 replay, regularization, constraint/isolation, growth/-compaction 으로 나눈 지도. 네 계열 모두 별도 retention 평가와 publication gate 가 필요하다.	23
12.1	Sleep controller의 강화학습 loop. 실제 보상은 다음 wake period의 utility·forgetting·비용에서 지연되어 관찰되므로 artifact lineage와 안전한 off-policy evaluation이 필요하다.	31
15.1	Inference, LoRA sleep, full fine-tuning sleep의 상대적 state stack. 실제 값은 optimizer, precision, sequence, checkpointing, sharding에 따라 달라지므로 이 그림은 구성요소 지도다.	38

List of Tables

1.1	학습 신호와 실제 제품 목표 사이의 간극	2
2.1	추론과 학습 단계의 대표 상태	5
3.1	Sleep update 의 평가 세트 예시	8
4.1	대표 training regime 의 상태와 비용	10
6.1	Sleep distillation destination 의 차이	15
7.1	Full FT, LoRA, adapter, external memory 의 비교	17
8.1	Wake event 를 sleep data 로 바꿀 때 필요한 gate	19
9.1	Sleep 관점의 continual-learning 비교 질문	24
10.1	Bounded memory 에서 쓰는 네 가지 capacity 동작	26
11.1	지도학습과 강화학습을 sleep control 에 적용할 때의 차이	29
12.1	Memory-policy reward 를 설계할 때 분리할 signal	32
13.1	배포 후 학습 상태의 시간척도	34
14.1	수정 · 삭제 destination 별 복구 성질	36
15.1	Sleep training job 의 주요 data movement	38
16.1	Sleep operator 의 공통 training/serving schema	40
G.1	Sleep-time compute 를 읽기 위한 canonical glossary	43

학습은 상태를 바꾸는 검증 가능한 절차다

1.1 추론과 학습을 먼저 분리하자

모델에 입력을 넣어 답을 얻는 과정과, 그 답을 바탕으로 모델 자체를 바꾸는 과정은 비슷해 보이지만 시스템 관점에서는 전혀 다르다. 데이터 x 와 현재 파라미터 θ 가 있을 때, 순전파는 현재 모델의 함수 f_θ 로 예측 $\hat{y} = f_\theta(x)$ 를 계산한다. 여기까지는 추론이다. 같은 입력과 같은 파라미터라면 원칙적으로 같은 상태에서 같은 결과를 얻는다.

학습은 한 단계 더 나아간다. 예측이 원하는 행동과 얼마나 다른지를 숫자로 만들고, 그 숫자를 줄이는 방향으로 θ 를 θ' 로 바꾼다. 따라서 학습의 최소 단위는 다음 상태 전환이다.

$$(D, \theta_t, S_t) \xrightarrow{\text{train}} (\theta_{t+1}, S_{t+1}, A_t), \quad (1.1)$$

여기서 D 는 데이터, S_t 는 optimizer 통계와 스케줄러 같은 보조 상태, A_t 는 checkpoint·로그·검증 결과 같은 산출물이다. “기억을 추가했다”는 표현만으로는 부족하다. 무엇이 바뀌었고, 어떤 데이터와 코드로 바뀌었으며, 다음 wake에서 어떤 버전이 활성화되는지까지 있어야 재현 가능한 학습이다.

정의 학습

학습은 관측 데이터와 명시된 목적을 사용해 모델 또는 외부 기억의 지속 상태를 바꾸고, 그 변화를 독립된 검증 데이터와 provenance로 판정하는 절차다. 단순히 긴 prompt를 읽거나한 응답 안에서 더 오래 생각하는 것은 지속 상태를 남기지 않는 한 이 정의의 학습이 아니다.

1.2 목적과 손실: 무엇을 좋아지게 할 것인가

목적함수는 학습 전체가 최적화하려는 양이다. 실제 구현에서는 한 예나 한 배치에서 계산하는 손실 ℓ 을 평균하거나, 여러 손실과 규제 항을 합쳐 목적 J 를 만든다.

$$J(\theta; D) = \frac{1}{|D|} \sum_{(x,y) \in D} \ell(f_\theta(x), y) + \lambda R(\theta). \quad (1.2)$$

R 은 가중치 크기, 이전 모델과의 차이, 안전 위반 같은 것을 벌점화할 수 있다. λ 는 현재 성능과 보존·안전 사이의 교환비다. 중요한 점은 “좋은 기억”이 하나의 자명한 목표가 아니라는 사실이다. 새 사실을 잘 답하게 하는 loss만 줄이면 오래된 사실, 말투, 안전 행동, tool 사용 능력이 함께 손상될 수 있다. Sleep-time training에서는 신규 유용성, 과거 보존, privacy, 삭제 가능성, compute 비용을 동시에 다뤄야 한다.

1.2.1 스칼라 예: 한 숫자를 맞추는 모델

가장 작은 모델 $f_w(x) = wx$ 를 생각하자. $x = 2$, 목표 $y = 10$, 현재 $w = 3$ 이면 예측은 6이다. 제곱 손실을

$$\ell(w) = \frac{1}{2}(wx - y)^2 \quad (1.3)$$

로 두면 현재 손실은 $\frac{1}{2}(6 - 10)^2 = 8$ 이다. w 를 조금 키우면 예측이 10 에 가까워진다. 이 단순한 “어느 쪽으로 움직일 것인가” 가 기울기의 직관이다. 하지만 실제 LLM 에서는 파라미터가 수십억 개이고, 한 변경이 수많은 입력의 출력에 영향을 준다. 그러므로 한 예에서 loss 가 줄었다는 사실은 기억이 올바르게 통합되었다는 증거가 아니다.

1.3 최적화 목표와 제품 목표는 다르다

학습 시스템은 흔히 proxy 를 최적화한다. next-token loss, preference score, retrieval hit-rate 는 계산할 수 있지만, 사용자가 몇 달 뒤 정확하고 적절한 도움을 받는지는 훨씬 늦게 관측된다. 이 간극은 sleep-time compute 에서 더 커진다. sleep job 의 출력이 오늘 응답에는 쓰이지 않고 다음 주의 다른 작업에서 쓰일 수 있기 때문이다.

Table 1.1: 학습 신호와 실제 제품 목표 사이의 간극

학습 신호	즉시 줄일 수 있는 것	놓칠 수 있는 것	필요한 검증
next-token loss	문장 예측 오차	사실성, 장기 일관성	held-out corpus, factual probe
preference reward	선호 데이터 적합도	reward hacking, minority preference	blind pairwise audit
retrieval hit	정답 문서 노출	과도한 context, 잘못된 사용	end-to-end task success
memory write success	저장 성공	later access, stale conflict	future-session replay

1.4 Sleep-time compute 와의 연결

Sleep phase 를 학습으로 부르려면 최소 여섯 질문에 답해야 한다.

1. 입력 데이터는 어느 wake interaction 에서 왔고, consent 와 filtering 을 통과했는가?
2. 교사 신호는 원문, 기존 모델, verifier, 사람 선호, 환경 reward 중 무엇인가?
3. 학생 상태는 full weights, LoRA, recurrent state, text/vector/graph 중 무엇인가?
4. 목적은 신규 유용성뿐 아니라 retention·privacy·cost 를 어떻게 포함하는가?
5. validation 은 업데이트에 사용하지 않은 미래형 task 를 포함하는가?
6. publish 와 rollback 은 어느 artifact version 을 원자적으로 활성화하는가?

핵심

학습 비용을 FLOPs 만으로 세면 부족하다. 데이터 읽기, activation 보관, gradient 와 optimizer state, checkpoint 쓰기, validation inference, artifact 이동, rollback 보존까지가 한 번의 학습 상태 전환 비용이다.

학습은 하나의 커널이 아니라 검증 가능한 상태 전환 파이프라인이다

Sleep-time compute도 동일한 파이프라인을 wake latency 바깥에서 수행한다.



Figure 1.1: 데이터에서 출판까지 이어지는 학습 상태 전환. Sleep-time compute 는 이파이프라인의 일부를 응답 latency 경로 밖으로 옮기지만 검증 단계를 없애지 않는다.

출처/제작: Original synthesis; context: SRC-STC-0030, SRC-STC-0014 대체 설명: 왼쪽의 데이터에서 오른쪽의 출판까지 일곱 상자가 화살표로 이어지고, 검증 실패 시 다시 데이터와 optimizer 로 돌아간다.

순전파, 역전파, optimizer 를 한 흐름으로 보기

2.1 순전파는 값을 만들고, 역전파는 책임을 나눈다

순전파는 layer 를 따라 activation 을 계산한다. 두 파라미터 모델 $f_{w,b}(x) = wx + b$ 에서 $x = 2$, $y = 10$, $w = 3$, $b = 1$ 이면 $\hat{y} = 7$ 이다. 제곱손실은 $\ell = \frac{1}{2}(7 - 10)^2 = 4.5$ 다. 출력 오차 -3 이 w 와 b 에 각각 얼마나 책임이 있는지 계산하는 과정이 역전파다.

기울기는 손실의 국소 경사다.

$$\frac{\partial \ell}{\partial w} = (wx + b - y)x = (-3) \cdot 2 = -6, \quad (2.1)$$

$$\frac{\partial \ell}{\partial b} = (wx + b - y) = -3. \quad (2.2)$$

음의 기울기는 해당 파라미터를 늘리면 손실이 줄어드는 국소 방향임을 뜻한다. 학습률 $\eta = 0.1$ 인 가장 단순한 gradient descent 라면

$$w' = w - \eta(-6) = 3.6, \quad b' = b - \eta(-3) = 1.3 \quad (2.3)$$

이고 새 예측은 8.5, 손실은 1.125 로 줄어든다. 이 예는 “기울기가 기억 그 자체” 가아님을 보여 준다. 기울기는 지금 본 예에 대한 국소 변경 제안이고, 실제로 어떤 상태를 얼마나 바꿀지는 optimizer 가 정한다.

2.2 Optimizer 는 update rule 과 기억 상태를 가진다

옵티마이저는 기울기를 파라미터 변화로 바꾸는 알고리즘이다. SGD 는 현재 기울기를 직접 쓰지만, momentum 은 과거 기울기의 이동평균을 유지한다. Adam 계열은 보통 1 차 모멘트 m_t 와 2 차 모멘트 v_t 를 유지한다. 학습률 η 는 한 update 에서 이 방향을 얼마나 크게 반영할지정하는 scale 이다.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (2.4)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (2.5)$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}. \quad (2.6)$$

m_t 와 v_t 가 옵티마이저 상태다. parameter 가 BF16 으로 2 byte 라 해도, 안정적인 학습을 위해 FP32 master weight, gradient, 두 moment 를 보관하면 학습 중 persistent state 는 parameter 당 12-16 byte 수준이 될 수 있다. sharding 과 8-bit optimizer 가 이를 줄일 수 있지만, “LoRA parameter 가 작다” 와 “sleep job 전체 memory footprint 가 작다” 는 동일한 문장이 아니다. activation 과 temporary buffer 가 별도로 존재하기 때문이다.

Table 2.1: 추론과 학습 단계의 대표 상태

상태	추론	학습	수명
model weights	읽기 중심	읽기+갱신	checkpoint 간 지속
activations	현재 token/layer	backward 까지 보존 또는 재계산	step 내부
gradients	없음	trainable parameter 마다 생성	accumulation window
optimizer state	없음	momentum/vari- ance/master copy	여러 step 지속
data state	prompt/KV	shuffle cursor, sampler, replay provenance	run 전체
validation artifact	선택적	metric, failure slice, approval	release 와 함께 지속

2.3 Activation memory 와 checkpointing

역전파는 각 layer 의 입력 activation 이 있어야 기울기를 계산하기 쉽다. 모든 activation 을 HBM 에 보관하면 빠르지만 memory 를 많이 쓴다. 일부를 버리고 backward 때 다시 순전파하는 activation checkpointing 은 compute 를 늘려 memory 를 줄인다. 따라서 같은 training FLOPs 라해도 sequence length, microbatch, layer checkpoint policy 에 따라 HBM 요구량과 wall-clock 이 달라진다.

Sleep-time training 은 wake serving 과 같은 GPU 에서 돌아갈 수 있으므로 peak memory 만이 문제가 아니다. checkpoint save 가 storage bandwidth 를 점유하고, parameter all-gather 가 interconnect 를 점유하며, validation 이 inference slot 을 가져간다. 좋은 스케줄러는 sleep job 의 평균 utilization 이 아니라 wake p99 latency 를 보호해야 한다.

2.4 Gradient accumulation 과 effective batch

한 번에 큰 batch 가 HBM 에 들어가지 않으면 여러 microbatch 의 gradient 를 더한 뒤 한 번 update 한다. accumulation step 을 K , microbatch 크기를 B_μ 라 하면 data-parallel replica 하나의 local effective batch 는 KB_μ 다. replica 가 N 개이고 각 replica 의 gradient 를 동기화하면 global batch 는 대략 NKB_μ 다.

하지만 token 길이가 다르면 “문장 수 batch” 와 “token 수 batch” 는 다르다. memory consolidation 에서는 긴 episode 몇 개가 gradient 와 activation 대부분을 만들 수 있다. 따라서 scheduler 는 examples/s 보다 tokens/s, effective valid memories/s, update 당 unique provenance 를 함께 기록해야 한다.

주의 작은 loss 가 곧 좋은 장기기억은 아니다

Training loss 가 내려가도 replay 샘플의 중복, contamination, teacher 오류를 외웠을 수 있다. 반대로 old-task loss 를 너무 강하게 고정하면 새 정보가 들어갈 공간이 사라진다. backward 는 정확히 계산할 수 있지만, 무엇을 backward 할지 정하는 목적과 데이터가 더 근본적인 문제다.

2.5 Sleep-time operator 로 번역하기

Parametric sleep job 을 한 번 실행한다고 하자. 먼저 selected episodes 를 forward 해 loss 를 계산하고, backward 로 gradient 를 만든 뒤 optimizer 가 adapter 또는 weights 를 갱신한다. 그 다음 held-out memory query, old-capability suite, privacy probe, latency benchmark 를 실행한다. 새 artifact 가 모두 통과하면 version pointer 를 바꾸고, 실패하면 이전 pointer 를 유지한다. 이 순서에서 학습 kernel 보다 중요한 시스템 계약은 다음과 같다.

1. source episode 와 gradient-producing example 사이의 provenance 가 유지될 것,
2. training state 가 wake serving state 와 원자적으로 분리될 것,
3. validation 전 artifact 가 사용자 traffic 에 노출되지 않을 것,
4. optimizer state 를 재개할지 폐기할지 명시할 것,
5. 삭제 요청 시 derived checkpoint 와 adapter 까지 추적할 수 있을 것.

배치, 샘플링, 검증: 데이터 흐름이 결과를 결정한다

3.1 Batch 는 단지 GPU 를 채우는 단위가 아니다

배치는 한 update 의 기울기를 추정하는 예시 집합이다. 전체 데이터의 평균 기울기를 매번 계산하는 대신 작은 subset 의 평균을 사용하면 계산은 줄지만 추정에 noise 가 생긴다. 적당한 noise 는 평평한 해를 찾는 데 도움이 될 수 있고, 지나친 noise 는 학습을 불안정하게 만든다. batch 를 키우면 noise 가 줄지만 activation memory 와 통신량이 늘고, 같은 epoch 에서 update 횟수가 줄어든다.

에폭은 데이터 전체를 대략 한 번 소비한 단위다. 고정 corpus 에서는 직관적이지만, 계속 유입되는 agent experience 에서는 명확한 epoch 가 없다. 이때는 processed tokens, unique episodes, wall-clock, user-days, consolidation cycles 를 별도로 기록해야 한다.

3.2 Sampling 은 무엇을 기억하고 무엇을 잊을지 정한다

샘플링은 다음 batch 에 들어갈 데이터를 정한다. 균등 sampling 은 흔한 사건을 반복 학습하고 rare but important event 를 거의 보지 못할 수 있다. recency sampling 은 freshness 를 높이지만 오래된 지식을 밀어낸다. surprise sampling 은 모델이 크게 틀린 경험을 강조하지만 adversarial outlier 와 noise 에 취약하다. value sampling 은 future reuse 가 높은 memory 를 선택하려 하지만 그 value 를 사전에 알기 어렵다.

Sleep system 의 admission score 를 개념적으로 다음처럼 둘 수 있다.

$$s_i = \alpha u_i + \beta n_i + \gamma r_i - \delta q_i - \epsilon p_i, \quad (3.1)$$

u_i 는 관측된 utility, n_i 는 novelty, r_i 는 expected reuse, q_i 는 품질 위험, p_i 는 privacy/governance 비용이다. 이것은 established formula 가 아니라 scheduler 가 숨겨 둔 판단을 명시적으로 드러내는 accounting form 이다. 계수는 offline replay 와 counterfactual evaluation 으로 검증해야 한다.

3.3 Training, validation, test 의 역할

검증 세트는 hyperparameter, checkpoint, stopping 시점을 고르는 데 사용한다. 테스트 세트는 선택을 마친 뒤 최종 일반화 성능을 추정한다. test 결과를 보고 계속 모델을 수정하면 test 도 사실상 validation 이 되고, reported score 는 낙관적으로 편향된다.

Memory system 에서는 split 단위가 더 어렵다. 같은 사용자의 같은 사건을 paraphrase 한 문장이 train 과 test 에 나뉘면 정보 누출이 생긴다. 공개 benchmark 를 검색할 수 있는 agent 는 test question 자체를 sleep memory 에 저장할 수 있다. 따라서 다음 split 을 함께 고려해야 한다.

- **time split**: 과거 episode 로 학습하고 미래 interaction 으로 평가한다.
- **entity split**: 특정 사람 · 프로젝트 · 도메인을 통째로 보존한다.
- **lineage split**: 동일 원문에서 파생된 summary 와 paraphrase 를 같은 split 에 둔다.
- **policy split**: admission/eviction 정책의 학습 trace 와 평가 trace 를 분리한다.
- **contamination audit**: benchmark 문항이나 정답이 검색 · 기억에 들어왔는지 검사한다.

3.4 Overfitting 과 generalization

과적합은 training data 에서 loss 가 낮지만 새로운 data 에서 성능이 나빠지는 현상이다. 일반화는 보지 못한 상황에서도 배운 규칙이 유효한 정도다. 개인 memory 에서는 “같은 사실을 정확히 다시 말하기”와 “그 사실을 새로운 작업에 적절히 적용하기”를 구분해야 한다. 전자는 retention, 후자는 transfer 에 가깝다.

장기 기억 평가는 최소 다섯 축이 필요하다.

1. **write**: 올바른 정보가 destination state 에 들어갔는가?
2. **access**: 관련 query 에서 그 정보가 실제 행동에 도달하는가?
3. **retention**: 시간이 지나고 다른 update 뒤에도 유지되는가?
4. **composition**: 다른 기억과 결합해 새로운 문제를 푸는가?
5. **governance**: source 를 추적하고 삭제 · 수정 · rollback 할 수 있는가?

Table 3.1: Sleep update 의 평가 세트 예시

세트	포함 내용	실패가 뜻하는 것	release gate
new-memory query	방금 통합한 사실 · 선호	write/access 실패	핵심 recall 하한
old-memory query	이전 cycle 의 기억	catastrophic forgetting	regression 한도
base capability	범용 지식 · 추론 · 안전	parametric side effect	non-inferiority
privacy/security	secret, poisoning, injection	영구 오염 위험	zero critical leak
systems	latency, bytes, energy, rollback	wake SLA/비용 실패	budget ceiling

3.5 통계적 유의성과 운영적 유의성

표본이 크면 작은 차이도 통계적으로 유의할 수 있다. 하지만 0.1 점 평균 향상이 adapter 수천 개, daily sleep cluster, 수십 TB write amplification 을 정당화하지는 않는다. 반대로 희귀하지만 치명적인 deletion failure 는 평균 score 에 거의 보이지 않는다. 따라서 effect size, confidence interval, tail risk, subgroup worst case, cost 를 함께 보고한다.

핵심

Sleep-time compute 의 올바른 test 는 “sleep 후 score 가 올랐는가” 하나가 아니다. 같은 future workload 에서 external retrieval, longer context, cache, periodic refresh, LoRA sleep 을 동일한 data budget 과 total cost 로 비교해야 한다.

배포 전·후의 학습 regime 지도

4.1 Regime 를 네 질문으로 구분한다

학습 이름이 많아 보이지만 네 질문으로 정리할 수 있다. (1) 언제 학습하는가, (2) 어떤데이터를 쓰는가, (3) 어떤 상태가 바뀌는가, (4) 무엇이 serving artifact 가 되는가. 이 네항목이 같다면 이름이 달라도 시스템 workload 는 비슷하고, 하나라도 다르면 compute 와 governance 가 달라진다.

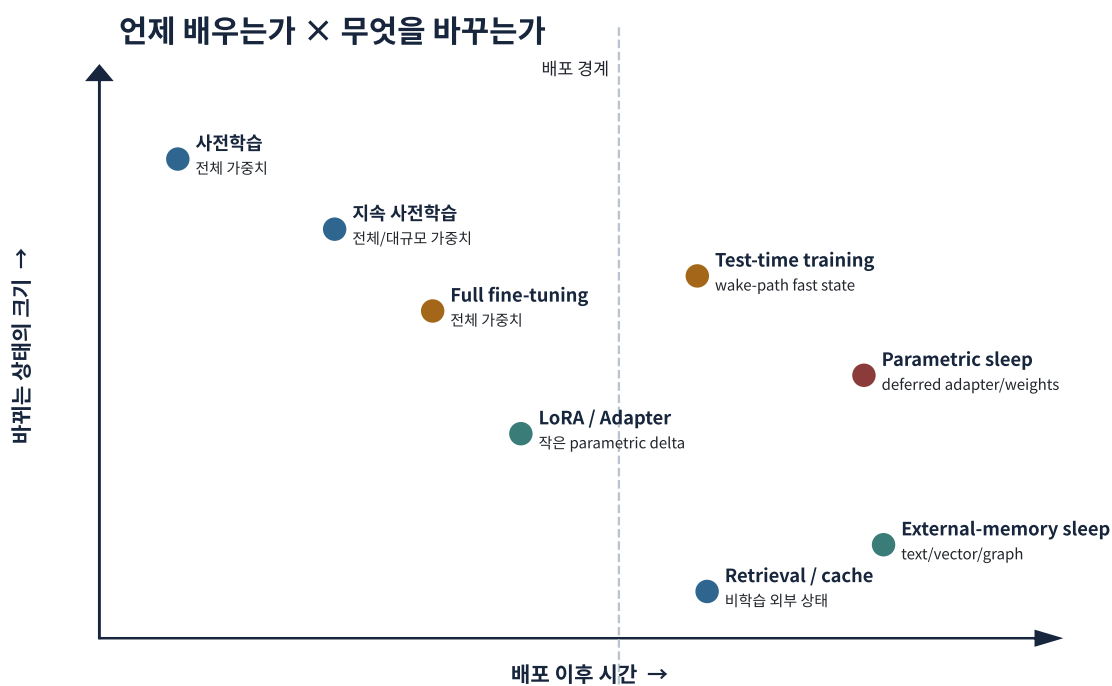


Figure 4.1: 배포 시점과 변경 상태 크기로 본 training regime. Test-time training 은 wake-path, sleep-time training 은 deferred path 에 놓이지만 둘 다 외부 기억과 parametric 상태를 선택할 수 있다.

출처/제작: Original synthesis; SRC-STC-0014, SRC-STC-0021, SRC-STC-0030 대체 설명: 가로축은 배포 이후 시간, 세로축은 변경 상태 크기이고 사전학습, fine-tuning, LoRA, TTT, parametric sleep, external-memory sleep 이 표시된다.

4.2 Pre-training 과 continued pre-training

사전학습은 대규모 corpus 에서 범용 representation 과 생성 능력을 형성한다. language model 은 보통 다음 token 의 negative log-likelihood 를 줄인다.

$$\mathcal{L}_{\text{LM}}(\theta) = - \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}). \quad (4.1)$$

한 번의 pre-training 은 data curation, tokenizer, distributed optimizer, checkpoint, evaluation 이 결합된 장기 run 이다. 모든 weight 와 optimizer state 가 sharding 되고, 장애 시재시작 가능한 checkpoint chain 이 필요하다.

지속 사전학습은 이미 훈련된 model 을 새 domain corpus 나 최신 corpus 로 계속 학습한다. 일반적으로 pre-training 과 같은 self-supervised objective 를 쓰지만 learning rate, data mixture, replay 비율을 조정한다. domain 지식은 늘 수 있으나 기존 능력과 언어 분포를 손상할 수 있어 base corpus replay 와 broad evaluation 이 필요하다. Global knowledge refresh 에는 유력하지만 per-user memory 를 매 interaction 마다 넣기에는 batching·privacy·rollback 단위가 너무 거칠 수 있다.

4.3 Full fine-tuning 과 instruction tuning

미세조정은 pretrained model 을 더 좁은 데이터나 행동 목표에 맞게 추가 학습한다. full fine-tuning 은 대부분 또는 모든 weight 를 trainable 하게 둔다. 작은 dataset 에서도 높은 표현력을 갖지만, gradient-optimizer state-checkpoint 가 full model scale 이고, 여러 사용자 variant 를 독립 보관하기 어렵다.

지시 튜닝은 instruction-response pair 로 원하는 task format 과 behavior 를 학습한다. 정답 token 에 대한 supervised loss 가 대표적이다. 답변 style 과 tool protocol 을 형성하기 좋지만, interaction log 를 그대로 정답으로 간주하면 사용자 오류와 assistant hallucination 을 재학습할 수 있다. Sleep dataset 은 raw transcript 가 아니라 verifier 를 통과한 target 과 negative example 을 포함해야 한다.

4.4 Preference optimization

선호 최적화는 두 출력 중 어느 것이 더 나은지에 대한 preference 를 사용한다. reward model 을 따로 학습하고 policy 를 RL 로 최적화할 수도 있고, chosen/rejected pair 에서 직접 loss 를 만들 수도 있다. 중요한 것은 algorithm 이름보다 reference behavior, preference provenance, distribution shift 다.

개인화 sleep 에서는 “사용자가 한번 수정했다” 는 사건을 영구 선호로 확대하면 위험하다. 상황적 요청, 장기 선호, 조직 policy 를 구분해야 한다. preference update 는 probability 를 바꾸기 때문에 사실 memory 와 달리 정확한 source sentence 하나로 rollback 하기 어렵다.

Table 4.1: 대표 training regime 의 상태와 비용

Regime	주 데이터	Trainable state	Serving artifact	주요 실패
pre-training	web/code/multi-modal corpus	전체 weights + optimizer	base checkpoint	bias, contamination, huge cost
continued PT	domain/latest corpus + replay	전체 또는 대규모 weights	refreshed checkpoint	domain drift, forgetting
full FT	labeled task/user data	전체 weights	full variant	interference, storage explosion
instruction tuning	instruction-response	전체 또는 PEFT	aligned checkpoint/adaptor	imitation of bad targets
preference opt.	comparisons/reward	policy + optional reward model	aligned policy	reward hacking, preference drift
LoRA/adaptor	task/user examples	작은 delta/module	base + delta	adaptor conflict, base churn
TTT	current input/stream	fast weights/state	session state	latency, instability
sleep training	accumulated episodes	chosen destination state	validated next version	stale data, rollback, fleet cost

4.5 Sleep-time 은 새로운 objective 가 아니라 lifecycle 축이다

Sleep-time compute 는 특정 optimizer 나 architecture 의 이름이 아니다. 같은 LoRA update 도 response 를 만들기 전에 current query 로 수행하면 test-time training 이고, response 이후 여러 episode 를 정리해 다음 session 의 adapter 를 만들면 sleep-time training 이다. 반대로 background process 가 text summary 와 graph edge 만 갱신한다면 training kernel 없이도 sleep lifecycle 에 속한다.

따라서 infra 를 분류할 때 “training cluster 인가 inference cluster 인가” 만 묻지 않는다. wake critical path, deferred transformation path, persistent state destination, validation and publication plane 을 따로 본다. 이 구분이 있어야 sleep job 을 spare GPU 에 단순 배치하는 접근과 state-aware lifecycle system 을 구별할 수 있다.

지도 신호, 자기지도, representation 과 언어모델 loss

5.1 정답은 어디서 오는가

지도 신호는 입력에 대해 원하는 target 이나 비교를 제공한다. image label, 정답 문장, tool execution result, 사람 preference 가 모두 supervision 이다. 정답 비용이 높고 누락·편향이 존재하므로 data provenance 가 중요하다.

자기지도학습은 raw data 안에서 target 을 만든다. 다음 token 예측은 앞 token 을 input, 실제 다음 token 을 target 으로 사용한다. 별도 사람이 모든 문장을 label 하지않아도 scale 할 수 있지만, 데이터가 사실인지 안전한지는 보장하지 않는다. Sleep transcript 를 그대로 next-token target 으로 쓰면 model error 와 prompt injection 도 학습 대상이 될 수 있다.

5.2 Representation 은 저장소와 다르다

표현은 input 을 후속 계산에 유용한 vector 또는 state 로 바꾼것이다. representation 안에 정보가 통계적으로 존재하는 것과, model 이 필요한 query 에서 그 정보를 꺼내 행동에 쓰는 것은 다르다. linear probe 가 fact 를 읽어도 generation path 가 접근하지 못할 수 있다. 그래서 parametric memory 의 capacity 는 단순 bit encoding, probe decodability, behavioral access, composition 을 나눠 측정해야 한다.

External vector memory 도 마찬가지다. embedding space 에 가까운 record 가 존재해도 retriever 가 top- k 에 넣지 못하거나, generator 가 context 를 무시하면 end-to-end memory 는 실패한다. Memory medium 에 상관없이 write, address, read, use 의 네 단계를 구분한다.

5.3 Logit, softmax, cross-entropy

언어모델의 마지막 layer 는 vocabulary 각 token 에 로짓 z_i 를 준다. 소프트맥스는 이를 확률로 바꾼다.

$$p_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}. \quad (5.1)$$

정답 token 이 y 일 때 교차엔트로피는 $\ell = -\log p_y$ 다. 정답 확률이 1 에 가까우면 0 에 가까워지고, 낮으면 커진다. 이 loss 는 모든 오답 token 을 동일한 의미로 보지 않는다. 확률 분포 전체를 통해 gradient 가 흐른다.

예를 들어 vocabulary 가 세 token 이고 logit 이 (2, 1, 0) 이면 softmax 확률은 대략 (0.665, 0.245, 0.090) 이다. 첫 token 이 정답이면 loss 는 약 0.408, 셋째가 정답이면 약 2.408 이다. 그러나 “서울은 한국의 수도” 를 맞히는 token loss 와 사용자 개인정보를 노출하는 token loss 는 제품 위험이 다르다. objective weighting 과 evaluation 이 이를 반영해야 한다.

5.4 Mask 와 sequence-level objective

LLM training 에서는 padding token, prompt token, tool output 중 어느 token 에 loss 를 줄지 mask 로 결정한다. Instruction tuning 에서 user prompt 까지 모방 loss 를 주면 user 발화를 assistant 가 생성하려 할 수 있다. tool trace 의 private field 를 mask 하지 않으면 secret 을 학습할 수 있다.

Token loss 만으로 sequence 의 factual consistency 나 task success 를 직접 표현하기 어렵다. verifier score, execution success, preference loss, contrastive loss 를 결합할 수 있지만 objective 가 많아질 수록 gradient conflict 와 weight selection 문제가 생긴다.

$$J = \lambda_{\text{new}} L_{\text{new}} + \lambda_{\text{replay}} L_{\text{old}} + \lambda_{\text{safety}} L_{\text{safety}} + \lambda_{\text{anchor}} D(f_{\theta}, f_{\theta_0}). \quad (5.2)$$

D 는 old model 과의 behavior 차이를 제한한다. 계수 하나를 바꾸면 무엇을 “기억” 으로 보존하는 지가 달라진다. Sleep controller 는 data selection 뿐 아니라 loss mixture 까지 policy 로 관리할 수 있지만, 그 policy 자체가 또 하나의 학습 · 검증 대상이 된다.

5.5 Systems 관점: data transformation 도 compute 다

Training FLOPs 이전에 transcript parsing, PII filtering, deduplication, semantic clustering, negative construction, teacher generation, verifier execution 이 있다. External-memory sleep 은 이 transformation 만 수행하고 weight backward 는 하지 않을 수 있다. 그래서 background synthesis 도 중요한 sleep-time compute 다. GPU FLOPs 가 작아도 graph update 와 vector index rewrite 가 storage-bound 일 수 있으며, source history 가 길수록 read amplification 이 커진다.

주의

Self-generated target 은 무료 data 가 아니다. teacher error, confirmation bias, model collapse, benchmark leakage 를 추적해야 한다. 실제 사용자/환경 관측이라는 anchor 와 recursive generation depth 를 provenance 에 남기지 않으면 나중에 품질 저하의 원인을 찾을 수 없다.

Distillation: 큰 경험을 작은 상태로 옮기기

6.1 교사와 학생

지식 증류는 교사 모델이 제공한 output distribution, rationale, feature, selected answer 를 학생 모델이 모방하도록 학습하는 방식이다. 교사가 더 크거나 더 많은 context 를 사용할 수 있고, 학생은 작거나 latency 가 낮을 수 있다. Sleep context 에서는 긴 episodic history 를 짧은 memory, adapter, 작은 model 로 압축하는 operator 로 이해할 수 있다.

Hard target 은 교사의 최고 확률 token 하나만 사용한다. Soft target 은 전체 분포를 사용해 두 번째, 세 번째 선택지에 대한 정보도 전달한다. 온도 T 를 적용하면

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \quad (6.1)$$

$T > 1$ 에서 분포가 평평해져 relative preference 가 더 드러난다. 학생 분포 q 와 교사 분포 p 사이 KL 발산을 줄일 수 있다.

$$D_{\text{KL}}(p||q) = \sum_i p_i \log \frac{p_i}{q_i}. \quad (6.2)$$

KL 은 대칭 거리가 아니다. 어떤 방향을 쓰는지에 따라 mode covering 과 mode seeking 특성이 달라진다. 실제 distillation loss 는 hard label loss 와 soft loss 를 섞는다.

$$L = \alpha L_{\text{hard}} + (1 - \alpha) T^2 D_{\text{KL}}(p_T || q_T). \quad (6.3)$$

6.2 Memory consolidation 으로서 distillation

긴 history 를 매번 prompt 에 넣는 대신 teacher 가 충분한 context 와 retrieval 을 사용해 reusable summary 나 strategy 를 만든다. destination 에 따라 세 종류로 나뉜다.

1. **External distillation:** episode 를 concise text, vector prototype, graph fact 로 바꾼다.
2. **Adapter distillation:** teacher 의 behavior 를 LoRA 나 adapter 에 학습한다.
3. **Model distillation:** 여러 cycle 의 memory 를 base model weights 로 통합한다.

첫 번째는 provenance 와 수정이 쉽고 inference 때 retrieval cost 가 남는다. 두 번째는 repeated query 의 retrieval cost 를 줄일 수 있지만 base model compatibility 와 adapter routing 이 필요하다. 세 번째는 access latency 가 가장 낮을 수 있으나 interference, deletion, rollback cost 가 가장 크다.

6.3 무엇이 압축 과정에서 사라지는가

Distillation 은 loss 가 중요하게 보는 behavior 만 보존한다. rare exception, uncertainty, source wording, provenance 는 teacher output 에 드러나지 않으면 사라진다. 외부 summary 가 “사용자는 항상 아침 운동을 선호한다” 고 일반화했지만 원 episode 가 “이번 주말” 이었다면 identity drift 가 발생한다.

세 가지 검증을 분리한다.

- **fidelity**: teacher 와 student 가 같은 query distribution 에서 얼마나 같은가?
- **utility**: teacher 와 달라도 downstream task 를 실제로 더 잘 푸는가?
- **provenance preservation**: output 의 각 claim 을 source episode 로 되돌릴 수 있는가?

Fidelity 가 너무 높으면 teacher 오류까지 복사한다. utility 만 최적화하면 근거 없는 shortcut 을 배울 수 있다. provenance 는 loss 한 향으로 쉽게 보장되지 않으므로 external metadata 와 release gate 가 필요하다.

6.4 Recursive distillation 과 model collapse

Sleep cycle k 의 output 이 cycle $k + 1$ 의 training data 가 되면 recursive depth 가 증가한다. 실제 data anchor 가 줄고 생성 data 만 반복되면 distribution tail 이 사라질 수 있다 (Shumailov et al. 2024). 반면 real data 를 계속 누적하는 조건에서는 collapse 가 필연적이지 않을 수 있다. 따라서 synthetic 여부의 이분법 대신 각 item 에 origin, teacher version, recursive depth, real-evidence links 를 기록한다.

Table 6.1: Sleep distillation destination 의 차이

Destination	update 비용	wake 비용	수정/삭제	대표 위험
text summary	낮음 – 중간	retrieval/context	쉬움	over-compression
vector prototype	중간	ANN search	중간	embedding drift
temporal graph	중간 – 높음	traversal + context	비교적 쉬움	conflict/edge explosion
LoRA/adapter	training 필요	routing + matmul	version rollback 가능	conflict/base churn
full weights	가장 높음	낮을 수 있음	가장 어려움	interference/un-learning

핵심

Distillation 은 기억을 “옮기는” 마법이 아니라 task distribution 이 정한 정보만 보존하는 lossy compression 이다. Sleep system 은 압축뿐만 아니라 삭제 가능한 provenance 와 later utility 를 함께 최적화해야 한다.

PEFT, LoRA, adapter, MoE: 얼마나 작은 상태를 바꿀 것인가

7.1 Trainable state 를 줄이는 이유

Full fine-tuning 은 모든 weight 에 gradient 와 optimizer state 를 만든다. 사용자 · 도메인별 variant 가 많아지면 compute 뿐 아니라 checkpoint storage 와 model residency 가 폭증한다. **PEFT** 는 base model 을 고정하고 작은 subset 또는 추가 module 만 학습해 이비용을 줄인다. 하지만 trainable parameter 비율과 total system cost 를 구분해야 한다. Base weight 는 여전히 forward/backward 에 참여하며 activation 과 input data 를 읽는다.

학습 가능 상태는 gradient 를 받아 갱신되는 상태다. 반면서빙 산출물은 실제 wake inference 에 배포되는 base checkpoint, delta, routing table, index 의 조합이다. Sleep job 이 작은 delta 만 만들더라도 수백만 사용자의 delta 를 catalog, cache, prefetch, revoke 하는 serving system 은 클 수 있다.

7.2 LoRA 의 최소 수식

LoRA 는 고정 weight $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$ 에 low-rank delta 를 더한다 (Hu et al. 2021).

$$W = W_0 + \Delta W, \quad \Delta W = \frac{\alpha}{r} BA, \quad (7.1)$$

$A \in \mathbb{R}^{r \times d_{in}}, B \in \mathbb{R}^{d_{out} \times r}$ 이고 $r \ll \min(d_{in}, d_{out})$ 다. full matrix 의 $d_{out}d_{in}$ 개 대신 $r(d_{in} + d_{out})$ 개를 학습한다. 예를 들어 $d_{in} = d_{out} = 4096, r = 16$ 이면 한 projection 의 trainable parameter 는 약 16.8M 에서 131K 로 줄어 약 128 배 작다.

그러나 다음 비용은 남는다.

- base layer 를 통과하는 forward 와 backward activation,
- LoRA A/B gradient 와 optimizer state,
- 여러 target module(Q/K/V/O, MLP) 에 대한 delta,
- user/domain 별 adapter 저장 · 인덱싱 · authentication,
- base checkpoint 가 바뀔 때 compatibility 재검증,
- 여러 adapter 를 합성할 때 발생하는 conflict.

7.3 Adapter module 과 routing

어댑터는 layer 사이에 작은 bottleneck module 을 추가한다. 전형적인 형태는

$$h' = h + W_{up} \sigma(W_{down} h) \quad (7.2)$$

이다. Base 를 고정할 수 있고 adapter 별 version 이 명확하다. 대신 inference graph 에 추가 operator 가 생기며, 어떤 adapter 를 언제 load 할지 routing 해야 한다. adapter 가 HBM 에 없으면 storage-to-HBM load latency 가 first-token latency 에 나타날 수 있다. 여러 작은 adapter 가 random access workload 를 만들면 총 byte 보다 IOPS 와 metadata lookup 이 병목일 수 있다.

7.4 Mixture of Experts 는 PEFT 와 다른 축이다

Mixture of Experts(MoE) 는 token 마다 router 가 일부 expert 만활성화하는 sparse architecture 다. 전체 parameter capacity 는 크지만 active FLOPs 는 일부다. MoE 가 자동으로 개인 memory system 이 되는 것은 아니다. Expert 가 pretrained/fixed 인지, sleep 중 새 expert 를 추가하는지, user-specific routing 을 학습하는지에 따라 의미가 달라진다.

Sleep-time 관점에서 MoE 는 세 기회를 준다. (1) stable domain knowledge 를 expert 로 격리, (2) base 전체가 아닌 selected expert 만 update, (3) cold expert 를 lower memory tier 에 offload. 동시에 all-to-all communication, expert imbalance, cache miss, expert proliferation 이라는 시스템 문제를 만든다.

Table 7.1: Full FT, LoRA, adapter, external memory 의 비교

방법	trainable state	optimizer state	wake access	rollback	핵심 병목
Full FT	전체 weight	전체 규모	직접 weight	checkpoint 단위	HBM/compute/interference
LoRA	low-rank delta	delta 규모	base+delta matmul	delta version	adapter catalog/cache
Adapter	작은 module	module 규모	extra module	module version	routing/load latency
MoE update	selected experts	expert subset	sparse routing	expert/version	all-to-all, imbalance
External memory	text/vector/graph	선택적	retrieval+con-text	record/version	index, context pollution

7.5 Selective promotion 이라는 hybrid

모든 episode 를 weight 로 넣지 않고 external system-of-record 에 먼저 보관한다. 충분한 future reuse, stability, cross-user generality, safety evidence 가 쌓인 item 만 adapter 나 periodic global refresh 로 승격한다. 이 구조는 raw episode 의 provenance 와 delete 를 유지하면서 high-reuse knowledge 의 repeated retrieval cost 를 줄일 수 있다.

Promotion break-even 의 최소 형태는 다음과 같다.

$$C_{\text{compile}} + C_{\text{validate}} + C_{\text{store}} < N_{\text{future}} (C_{\text{retrieve}} - C_{\text{parametric access}}). \quad (7.3)$$

N_{future} 가 작거나 base model churn 이 빠르면 승격은 손해다. 따라서 adapter 는 “항상 더 효율적인 기억” 이 아니라 later reuse 가 충분할 때만 이기는 compiled cache 에가깝다.

주의 작은 **delta** 의 함정

LoRA file 이 작다는 사실만으로 user-specific parametric memory 가 scale 한다고 결론 내릴 수 없다. adapter 수, active working set, load pattern, validation cost, base-version migration, deletion SLA 를 함께 측정해야 한다.

Sleep 이 먹는 것: data curation, augmentation, replay

Sleep-time compute 가 training 이라면 첫 질문은 “어떤 optimizer 를 쓸까” 가 아니라 “어떤 경험을 학습 가능한 증거로 만들까” 다. Wake log 에는 사용자 발화, model response, tool result, correction, 반복 조회, 실패 trace 가 섞여 있다. 이들을 그대로 next-token data 로 넣으면 개인 식별 정보, 틀린 model output, 중복된 쉬운 예, 시간 순서가 깨진 사실이 모두 같은가중치로 학습된다. 따라서 sleep pipeline 의 품질 상한은 대개 data plane 에서 정해진다.

8.1 Log 와 training example 은 다르다

데이터 큐레이션은 후보 경험을 수집하고 정규화하며 중복을 제거하고, 목표 · 권리 · 민감도 · 시간 범주를 부여하는 전 과정이다. 데이터 필터링은 그중 정해진 기준으로 남길 것과 제외할 것을 가르는 단계다. 둘을 분리해야 “수집은 했지만 학습에는 쓰지 않음”, “외부 기억에는 보존하지만 parametric update 에는 금지” 같은 결정을 표현할 수 있다.

한 sleep example e_i 를 최소한 다음 record 로 본다.

$$e_i = (x_i, y_i, t_i, u_i, s_i, q_i, r_i, p_i, v_i), \quad (8.1)$$

여기서 x_i 는 입력 context, y_i 는 목표 또는 preference, t_i 는 event time, u_i 는 user/tenant scope, s_i 는 source type, q_i 는 quality 와 uncertainty, r_i 는 retention/rights, p_i 는 provenance pointer, v_i 는 생성한 model-tool version 이다. 실제 시스템에서는 원문과 이 record 를 분리해 저장할 수 있다. 원문 삭제 요청을 처리하려면 record 가 어느 artifact 와 update 에 기여했는지도 이어져야 한다.

출처 추적성은 단순 URL 열이 아니다. “원본 event → 정제 record → replay batch → checkpoint → serving version” 을 역추적할 수 있어야 한다. provenance 가 없으면 잘못된 기억을 발견해도 external index 한 row 만 지우면 되는지, LoRA 를 폐기해야 하는지, base model 을 다시 만들어야 하는지 판단할 수 없다.

Table 8.1: Wake event 를 sleep data 로 바꿀 때 필요한 gate

Gate	묻는 질문	실패 시 기본 동작
scope	누구의 기억이며 어느 tenant 에서만 유효한가?	global 학습 금지, 격리 저장
grounding	correction 이나 fact 가 신뢰 가능한 source 에 연결되는가?	외부 후보로 격하 또는 제외
temporal	언제부터 언제까지 참이며 newer record 와 충돌하는가?	time interval 과 supersession edge 기록
rights/privacy	저장 · 재생 · 가중치 반영 · 장기 보존 권한이 각각 있는가?	허용된 destination 만 사용
difficulty	모델이 이미 맞는 중복 easy example 인가?	down-sample 또는 대표 예만 보존
contamination	validation/test 또는 비공개 benchmark 정답이 섞였는가?	quarantine, 평가 집합 재구성

8.2 Augmentation 은 의미를 보존한다는 가정이다

데이터 증강은 한 예시를 여러 변형으로 늘리는 기법이지만, 핵심은 “이 변환 아래 목표가 변하지 않는다” 는 invariance 가정이다. 이미지 회전처럼 명확한경우도 있지만 memory data 에서는 더 위험하다. 발화를 paraphrase 하면 표현 다양성은 늘지만 negation, 날짜, actor, uncertainty 가 바뀔 수 있다. context 를 잘라내면 정답은 남아 보여도근거와 privacy scope 가 사라질 수 있다.

Sleep 용 augmentation 은 세 층으로 나누면 안전하다.

1. **surface augmentation**: 표현 · 순서 · format 을 바꾸되 entity, time, polarity, provenance 를 보존한다.
2. **contrast construction**: 헛갈리는 old/new fact, 같은 이름의 다른 사람, 허용/금지 scope 를 쌍으로 만들어 decision boundary 를 학습한다.
3. **counterfactual probe**: 목표 자체를 학습시키기보다 모델이 evidence 없이외운 답을 내는지 평가한다. training set 과 validation set 을 섞지 않는다.

합성 데이터는 teacher model, simulator, rule 로 만든 예시다. 긴 episode 를 question-answer pair 로 변환하거나 드문 failure 를 증폭하는 데 유용하지만, 생성자의 오류와 표현 편향도 함께 복제한다. recursive distillation 에서 생성 data 가 다시 다음 cycle 의 teacher data 가 되는 깊이를 record 하고, real-evidence anchor 를 일정 비율 유지해야 하는 이유가 여기에 있다 (Shumailov et al. 2024).

8.3 Replay: 과거를 다시 보여 주는 가장 직접적인 기준선

리플레이 버퍼는 과거 경험 일부를 다시 학습시키기 위한 제한된 저장소다. 새 batch B_{new} 만 학습하지 않고 replay sample B_{old} 를 섞으면

$$\mathcal{L}_{sleep} = \lambda_{new}\mathcal{L}(B_{new}) + \lambda_{old}\mathcal{L}(B_{old}) + \lambda_{safe}\mathcal{L}(B_{safety}) \quad (8.2)$$

처럼 plasticity 와 retention 을 직접 trade-off 할 수 있다. 작은 episodic memory 도 forgetting 을 줄이는 강한 baseline 이 될 수 있으며 (Chaudhry et al. 2019), class prototype 에 가까운대표 예를 유지하는 방식도 쓰인다 (Rebuffi et al. 2017). 그러나 buffer 가 raw episode 를 보존하면 저장 · privacy 비용이 크고, random sampling 은 다수 사용자의 반복 패턴이 희귀하지만중요한 경험을 밀어낸다.

코어셋은 전체 데이터의 loss 나 representation 을 작은 weighted subset 으로 근사하려는 관점이다. Sleep system 에서는 단순 diversity 외에도 다음 utility 를 함께 본다.

$$U(e) = \alpha N(e) + \beta E(e) + \gamma F(e) + \delta R(e) - \eta C_{store}(e) - \zeta C_{risk}(e), \quad (8.3)$$

N 은 novelty, E 는 evidence quality, F 는 예상 future reuse, R 은 forgetting 방지 기여다. 이 점수는 established formula 가 아니라 controller 를 설계하기 위한 decomposition 이다. 각 항의 scale 과 label delay 가 다르므로 하나의 숫자로 만들기 전에 raw features 와 의사결정 이유를 남긴다.

생성 리플레이는 raw 과거를 전부 저장하는 대신 generator 가 과거분포를 복원하도록 한다 (Shin et al. 2017). 저장 bytes 를 줄일 수 있지만 generator 자체도있고, 원본 provenance 를 잃으며, tail 을 평탄화할 수 있다. 따라서 “raw replay 대 generated replay” 는 어느 하나를 고르는 문제가 아니라 민감한 원문, 검증된 canonical fact, 일반 skill pattern 에 서로 다른 retention 을 주는 tiering 문제다.

핵심 Sleep dataset 의 최소 산출물

Sleep job 입력은 folder 하나가 아니라 immutable dataset manifest 여야 한다. manifest 에는 source snapshot, filter version, included/excluded count 와 reason, tenant/time scope, dedup cluster, augmentation lineage, train/validation split, hash, expiry 가 포함된다. 같은 checkpoint 를 다시 만들 수없다면 rollback 도 감사도 불가능하다.

8.4 평가 누수와 시간 누수

개인화 memory 에서는 random split 이 특히 위험하다. 같은 episode 의 paraphrase 가 train 과 validation 에 나뉘면 memorization 을 generalization 으로 오해한다. 미래 correction 이 과거 시점 평가 입력에 들어가면 temporal leakage 가 생긴다. 권장 split unit 은 token 이 아니라 user, episode, entity thread, time window 다. 평가는 “현재 사실 회상”, “과거 시점 질의”, “다른 user 로의 비누출”, “삭제 후 비회상” 을 분리한다.

Sleep-time compute 에서 data volume 은 공짜 scaling 축이 아니다. 중복 experience 를 더 많이 replay 하면 FLOPs 는 늘지만 effective evidence 는 거의 늘지 않는다. 그래서 future scaling law 의 독립변수는 raw token 수보다 provenance-weighted novel evidence, replay diversity, recursive depth, validation coverage 에 가까워야 한다.

계속 배우면서 잊지 않기: continual learning의 문제 설정

지속학습은 데이터 분포가 한 번에 주어지지 않고 시간에 따라 도착할 때, 새 능력을 획득하면서 과거 능력도 유지하려는 설정이다. Sleep-time compute가 매일 또는 매 session 뒤모델을 갱신한다면 자연스럽게 continual learning 문제가 된다. 단, 실제 agent stream은 benchmark 처럼 “Task 1 종료, Task 2 시작” 표지를 주지 않으며 사용자·사실·도구·정책 변화가 겹친다.

9.1 왜 새 학습이 과거 능력을 지우는가

한 파라미터 θ_j 가 과거 task A와 새 task B에 동시에 쓰인다고 하자. B의 loss를 줄이는 gradient가 A의 loss를 늘리는 방향이면 같은 update가 양쪽에 반대 효과를 낸다. 새 data만 반복하면 optimizer는 과거 data를 보지 못하므로 그 손실 증가를 감지하지 못한다. 이를 **파국적망각**이라 부른다. “parameter capacity가 꽉 찼다”는 말과 같지는 않다. 충분한 파라미터가 있어도 optimization path와 sampling 때문에 잊을 수 있고, 반대로 작은 모델도 잘 선택한 replay와 isolation으로 오래 유지할 수 있다.

안정성-가소성 dilemma는 두 오류를 함께 본다.

- 안정성만 높이면 이전 기능은 보존하지만 새로운 correction과 변화에 적응하지 못한다.
- 가소성만 높이면 최신 session에는 잘 맞지만 오래된 skill과 safety behavior가 무너진다.

Complementary Learning Systems 관점은 빠른 episodic store와 느린 cortical integration을 분리하고, interleaved replay로 새 경험을 기존 구조에 통합한다 (McClelland et al. 1995). 이는 biological sleep을 그대로 구현하라는 처방이 아니라, fast store가 제공하는 sparse episode와 slow learner가 요구하는 mixed training distribution이 다르다는 설계 basis다.

9.2 성능 행렬로 retention과 transfer를 분리한다

$R_{i,j}$ 를 task j 까지 학습한 뒤 task i 에서 측정한 성능이라 하자. 마지막 시점 평균 성능은

$$A_T = \frac{1}{T} \sum_{i=1}^T R_{i,T} \quad (9.1)$$

이고, task i 가 과거에 기록한 최고 성능에서 마지막 성능이 떨어진 양은

$$F_T = \frac{1}{T-1} \sum_{i=1}^{T-1} \left(\max_{j \in \{i, \dots, T-1\}} R_{i,j} - R_{i,T} \right) \quad (9.2)$$

로 측정할 수 있다. **후방 전이**는 새 학습이 과거 task를 개선하면 양수, 악화하면 음수인 방향성 지표다 (Lopez-Paz and Ranzato 2017). 평균 정확도 하나는 latest adaptation과 retention을 섞으므로 sleep controller 비교에 부족하다.

LLM memory 에는 task label 대신 evaluation slice 를 둔다. 예를 들어 user preference, stable personal fact, time-varying fact, tool procedure, safety refusal, general capability, privacy non-leakage 를 별도행으로 만든다. 각 sleep version 뒤 이 행렬을 다시 측정해야 어느 축에서 얻고 잃었는지 보인다.

안정성-가소성 문제를 푸는 네 가지 레버

어느 한 방법도 용량·재현·롤백·보안을 동시에 해결하지 않는다.

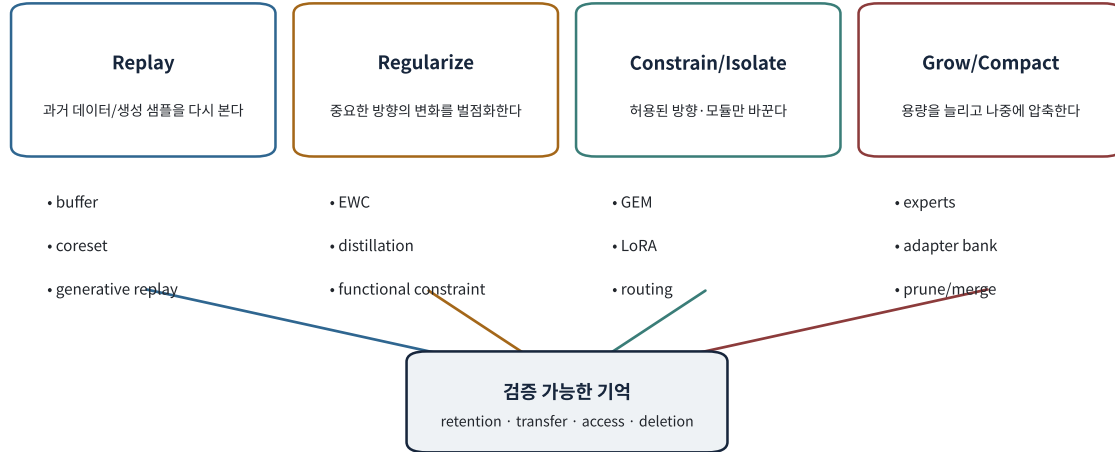


Figure 9.1: Continual-learning 방법을 replay, regularization, constraint/isolation, growth/compaction 으로 나눈 지도. 네 계열 모두 별도 retention 평가와 publication gate 가 필요하다.

출처/제작: Original synthesis; SRC-STC-0002, SRC-STC-0003, SRC-STC-0004, SRC-STC-0007 대체 설명: 왼쪽의 네 방법 계열이 중앙의 후보 상태로 모이고, 평가와 출판을 통과해야 장기 기억으로 승격되는 흐름도.

9.3 네 가족의 방법과 숨은 비용

1. **Replay:** 과거 data 또는 output 을 다시 보여 준다. 가장 직접적이지만 storage, sampling, privacy 와 data movement 비용을 낸다.
2. **Regularization:** 과거에 중요한 parameter 가 움직일 때 loss penalty 를 준다. raw replay 를 줄일 수 있지만 중요도 state 와 task boundary 가정을 요구한다.
3. **Constraint/isolation:** 과거 loss 를 나쁘게 하는 gradient 를 투영하거나 parameter subspace 를 분리한다. interference 는 줄지만 routing 과 fragmentation 이 생긴다.
4. **Growth/compaction:** 새 module 을 추가한 뒤 prune, merge, distill 한다. plasticity 는 높지만 capacity 가 단조 증가하지 않도록 별도 budget controller 가 필요하다.

9.4 온라인 평가와 shadow publication

Sleep update 직후 사용자에게 바로 노출하면 과거 failure 를 되돌릴 관찰 기간이 없다. 후보 version 은 offline replay, held-out temporal test, adversarial scope test 를 통과한 뒤 shadow traffic 에서 current version 과 비교한다. 사용자별 adapter 라 해도 global safety behavior 와 다른 tenant non-leakage 를 함께 검증한다. rollback unit 은 optimizer step 이 아니라 dataset manifest 와 candidate artifact 전체다.

Table 9.1: Sleep 관점의 continual-learning 비교 질문

계열	추가로 저장하는 것	주요 compute/I/O	실패 질문
replay	raw/summary/example, metadata	storage read, forward/backward	누가 buffer 에서 밀려났는가?
regularization	old weights, importance	penalty compute, state read	중요도 추정이 새 domain 에도 맞는가?
projection	episodic gradient/reference	gradient dot product/-solve	제약이 plasticity 를 봉쇄하는가?
isolation/-growth	mask, adapter, expert, router	routing, fragmented reads	module 수와 cold state 가 폭증하는가?
distill/compact	teacher snapshot, student	teacher inference + training	tail-provenance 가 사라졌는가?

주의 Benchmarks 의 task boundary 를 그대로 믿지 말 것

실서비스에서는 분포 변화, 사용자 correction, 일시적 event, 공격 input 이 섞인다. task ID 를 알고 있는 EWC/GEM 성능만으로 sleep system 을 선택하면 trigger detection 과 data contamination 이라는 실제 비용을 누락한다. 연구 보고에는 boundary oracle 사용 여부와 buffer budget 을 반드시 적는다.

Continual learning 은 “무한히 기억한다” 는 약속이 아니다. 무엇을 잊어도 되는지, 어느 fidelity 로 압축할지, 어떤 능력은 immutable base 에 남길지 명시하는 bounded optimization 이다. 다음 장은 그 mechanism 과 capacity 한도를 수식으로 연결한다.

보존·격리·확장·압축: 안정성-가소성 mechanism

이 장은 continual learning 의 대표 도구를 “무엇을 저장하고 어떤 update 를 금지하는가” 로 설명한다. 알고리즘 이름보다 중요한 것은 data, auxiliary state, constraint, capacity 회수 경로다.

10.1 Regularization: 중요한 parameter 는 덜 움직인다

정규화는 새 task loss 에 보존 조건을 더한다. EWC 는 과거 parameter θ_i^* 와 importance F_i 를 저장하고 다음 objective 를 사용한다 (Kirkpatrick et al. 2017).

$$\mathcal{L}_{EWC}(\theta) = \mathcal{L}_{new}(\theta) + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^*)^2. \quad (10.1)$$

EWC 의 직관은 F_i 가 큰 parameter 를 “과거에 중요하므로 비싸게 움직이는 spring” 으로 만드는 것이다. raw replay 가 없어도 되지만 매 cycle 마다 reference weights 와 importance 를 보존해야 한다. task 가 늘수록 importance 를 합칠지, 최근 것만 둘지, 충돌하는 중요도를 어떻게 낮출지 결정해야 한다. LLM 전체 weight 에 per-parameter FP32 importance 를 두면 state 가 model size 만큼 늘 수 있으므로 layer/block/low-rank 단위 근사가 필요하다.

10.2 Gradient constraint: 과거 손실을 늘리는 방향을 피한다

새 gradient g 와 replay example 의 과거 gradient g_k 가 있을 때 $g^\top g_k < 0$ 이면 두 objective 가 국소적으로 충돌한다. 기울기 투영은 update 를

$$\tilde{g} = \arg \min_z \frac{1}{2} \|z - g\|_2^2 \quad \text{s.t.} \quad z^\top g_k \geq 0 \quad \forall k \quad (10.2)$$

같은 feasible region 으로 옮긴다. GEM 은 episodic memory 와 이 제약을 결합해 negative backward transfer 를 줄이고 positive transfer 도 측정했다 (Lopez-Paz and Ranzato 2017). 그러나 reference gradient 를 많이 둘수록 backward 와 projection solve 비용이 커진다. Sleep cluster 에서 compute 를 쓸 수 있어도, 사용자 수에 비례한 gradient memory 를 HBM 에 올리는 것이 병목일 수 있다.

10.3 Isolation 과 growth: 충돌할 parameter 를 나눈다

파라미터 격리는 base 를 freeze 하고 user/task adapter, mask, expert 를 따로 둔다. locality 와 rollback 이 쉽고 global forgetting 을 줄이지만, active artifact 수가 늘며 routing metadata 와 random read 가 증가한다. 같은 pattern 을 여러 adapter 가 중복 학습하면 logical capacity 보다 physical capacity 가 빠르게 소진된다.

동적 확장은 새 분포가 기존 subspace 로 충분히 표현되지 않을 때 module 을 추가한다. 이는 새 data 를 강제로 기존 weights 에 넣는 것보다 plastic 하지만, append-only 라면 결국 capacity 를 초과한다. expansion 조건에는 validation gain 뿐 아니라 expected reuse 와 bytes-latency 가 들어가야 한다.

반대 방향의 **프루닝**은 importance 가 낮은 weight, expert, memory item 을 제거한다. Parameter magnitude 만 보면 희귀하지만 critical 한 기억을 지울 수 있다. Sleep system 의 prune unit 은 “작은 수치” 보다 provenance cluster, last verified time, future query utility, replacement coverage 를 포함해야 한다.

10.4 Capacity 는 bytes 만이 아니라 interference 와 운영 한도다

용량 예산 B 를 단순 저장 bytes 가 아니라 다음 resource vector 로 둔다.

$$\mathbf{B} = (B_{\text{bytes}}, B_{\text{latency}}, B_{\text{energy}}, B_{\text{risk}}, B_{\text{ops}}). \quad (10.3)$$

B_{ops} 는 version 수, validation 시간, rollback 복잡도 같은 운영 capacity 다. 외부 vector store 가 남은 SSD bytes 를 충분히 가져도 ANN latency 와 index rebuild 가 먼저 한도에 닿을 수 있다. LoRA file 이작아도 사용자 수가 수억이면 adapter lookup 과 batching fragmentation 이 한도다.

압축은 허용 왜곡 안에서 representation cost 를 줄인다. 여러 episode 를 summary 로, 여러 vector 를 prototype 으로, 여러 adapter 를 merged adapter 로 바꿀 수 있다. **율-왜곡**관점에서는 memory representation Z 의 rate $R(Z)$ 와 future query distribution 에서의 distortion $D(X, \hat{X})$ 를 함께 최적화한다.

$$\min_{q(z|x)} \mathbb{E}[D(X, \hat{X}(Z))] + \beta R(Z). \quad (10.4)$$

여기서 D 는 원문 reconstruction loss 일 필요가 없다. future task success, factual calibration, provenance recovery, deletion granularity 를 함께 반영해야 한다. 이 framework 를 memory compaction 에 직접 연결하는 연구가 등장하고 있으나 (Anonymous 2026), 어떤 distortion 이 장기 agent utility 를 대변하는지는 아직 열린 측정 문제다.

10.5 Admission, eviction, consolidation 은 하나의 정책이다

기억 입장 정책은 새 event 를 discard, raw log, text summary, vector, graph, adapter candidate, global training queue 중 어디로 보낼지 정한다. **기억퇴거 정책**은 예산이 부족할 때 delete, compress, demote, merge 중 하나를 고른다. 둘을 따로 최적화하면 새 item 을 받아 놓고 즉시 비슷한 old item 을 지우는 thrashing 이 생긴다.

공고화는 여러 경험을 더 안정적인 표현으로 결합하는 전체 변환이다. 중요한점은 destination 이 항상 model weights 가 아니라는 것이다. episode $\rightarrow$ temporal graph, vector cluster $\rightarrow$ prototype, repeated correction $\rightarrow$ LoRA candidate 처럼 evidence 와 reuse 에 따라 경로가 달라진다.

Table 10.1: Bounded memory 에서 쓰는 네 가지 capacity 동작

동작	얻는 것	잃을 수 있는 것	필수 검증
delete	즉시 bytes-rights 회수	future utility, tail evidence	삭제 대상 · 파생물 완전성
compress	lower tier 비용 감소	세부 · 불확실성 · provenance	distortion slice 별 회귀
merge/distill	중복 compute 감소	충돌 사실, minority behavior	locality 와 conflict test
promote	wake latency 감소 가능	weight interference, rollback 비용	retention·safety·break-even

핵심 단조 증가를 막는 invariant

각 sleep cycle 은 새 bytes 만 보고하지 말고 ΔB 를 destination 별로 보고해야 한다. admission 된상태가 있으면 eviction·compression·reuse gain 중 무엇이 그 비용을 정당화하는지 연결한다. “모두보관 후 나중에 정리” 는 retention policy 가 아니라 unbounded debt 다.

강화학습의 최소 토대: sleep 을 하나의 의사결정 문제로 보기

강화학습 (Reinforcement Learning, RL) 은 “정답 label 을 바로 주기 어렵지만 행동 뒤 결과는 관찰할 수 있는” 순차 의사결정 문제를 다룬다. Sleep-time compute 에서 RL 의 주된 역할은 language model 전체를무조건 RL 로 다시 학습하는 것이 아니라, 언제 무엇을 저장 · 압축 · 승격할지 정하는 controller 를 학습하는데 있다.

11.1 State, action, reward, trajectory

RL 상태 s_t 는 시점 t 에서 정책이 볼 수 있는 정보다. 예를 들어 buffer fill ratio, 새 correction 수, retrieval miss, user inactivity window, available GPU, artifact age, privacy class 가 state feature 가 된다. state 에 미래 정보나 evaluation answer 가 들어가면 leakage 다.

행동 a_t 는 “아무것도 하지 않기”, summarize, re-embed, graph merge, replay batch 생성, LoRA training, candidate validate, promote, rollback 처럼 시스템을 바꾸는 선택이다. action 은결과뿐 아니라 compute reservation, data read, model lock 이라는 비용을 가진다.

보상 r_t 는 행동의 즉시 결과다. 단순 accuracy gain 만 쓰면 controller 는 비싼 full fine-tuning 을 자주 고르거나, 쉬운 query 를 암기해 점수를 올릴 수 있다. 그래서 reward 를

$$r_t = \Delta U_t - \lambda_c C_t - \lambda_l L_t - \lambda_r Risk_t - \lambda_f Forget_t \quad (11.1)$$

처럼 utility 증가, compute/energy cost, wake latency, risk, forgetting 으로 분해한다. 계수는 moral truth 가아니라 운영 정책이므로 버전과 승인 주체를 기록한다.

궤적 $\tau = (s_0, a_0, r_0, s_1, \dots)$ 는 여러 시점의 순서다. 한 summary 의 utility 는다음날 retrieval 에서, 한 adapter 의 부작용은 수주 뒤 rare slice 에서 드러날 수 있다. 따라서 즉시 reward 만보면 장기 효과를 놓친다.

11.2 Return, policy, value

리턴은 미래 보상의 할인합이다.

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad 0 \leq \gamma < 1. \quad (11.2)$$

γ 가 작으면 오늘의 compute 절약을, 크면 오래 뒤 retention 과 maintenance 를 더 본다. 삭제처럼 되돌리기 어려운 action 에는 단일 discount factor 보다 explicit safety constraint 와 long-horizon audit 가필요하다.

정책 $\pi(a|s)$ 는 state 에서 action distribution 을 정한다. rule-based threshold 도 policy 이고 neural controller 도 policy 다. 먼저 rule baseline 을 두어야 learned policy 가 무엇을 개선했는지알 수 있다. **가치 함수** $V^\pi(s) = \mathbb{E}[G_t | s_t = s]$ 는 state 의 예상 장기성과를, $Q^\pi(s, a)$ 는 action 까지 정했을 때의 성과를 추정한다.

신용 할당은 지연된 성공·실패를 어떤 과거 action 에 돌릴지 정하는 문제다. 한 달 뒤 사용자가 오래 된 preference 를 정확히 회상했을 때 어느 nightly consolidation 이 기여했는지 알기 어렵다. artifact lineage 와 counterfactual shadow evaluation 이 없으면 reward label 자체가 noisy 해진다.

11.3 Policy gradient 와 update 가 실제로 의미하는 것

정책 기울기는 기대 return $J(\phi) = \mathbb{E}_{\tau \sim \pi_\phi}[G_0]$ 을 높이는 controller parameter ϕ 의 방향을 추정한다.

$$\nabla_\phi J(\phi) = \mathbb{E}[\nabla_\phi \log \pi_\phi(a_t|s_t) A_t], \quad (11.3)$$

A_t 는 action 이 평소 예상보다 얼마나 좋았는지 나타내는 advantage 다. PPO 는 한 batch 를 여러 epoch 사용하되 policy 가 한 번에 너무 멀리 움직이지 않도록 clipped surrogate objective 를 사용한다 (Schulman et al. 2017). 이름을 외우기보다 “rollout 수집 → advantage 추정 → policy update → 새 policy 로 다시 수집” 이라는 dependency 를 이해해야 한다.

11.4 On-policy 와 off-policy 의 systems 차이

온폴리시 학습은 현재 policy 가 만든 experience 를 사용한다. distribution mismatch 는 작지만 새 policy 마다 실제 또는 simulator rollout 이 필요하고, unsafe action 을 exploration 할 위험이 있다. 오프폴리시 학습은 과거 policy 나 rule controller 가 만든 log 도 사용한다. 기존 운영 trace 를 재활용 할 수 있지만, 과거에 거의 선택되지 않은 action 의 결과는 데이터에 없다는 support 문제가 생긴다.

Sleep controller 에서는 실제 user data 로 무작정 exploration 하지 않는다. 안전한 순서는 (1) trace-driven simulator, (2) counterfactual cost model, (3) shadow execution, (4) 제한된 canary, (5) gradual rollout 이다. “do nothing” 과 현재 heuristic 은 항상 baseline action 으로 남긴다.

Table 11.1: 지도학습과 강화학습을 sleep control 에 적용할 때의 차이

항목	지도/선호 분류	순차 RL
label	이 state 에서 어떤 action 이 좋은가	action 뒤 여러 시점의 reward
data	독립 예시로 근사 가능	policy 가 방문한 trajectory distribution
강점	안정적이고 offline 검증이 쉬움	지연 효과와 action sequence 최적화 가능
위험	expert label 비용, myopic target	exploration, reward hacking, counterfactual 부재
권장 시작	heuristic imitation, pairwise ranking	simulator 와 shadow 에서 제한적으로

주의 Reward 가 곧 요구사항은 아니다

측정 가능한 proxy 를 reward 로 쓰면 controller 는 proxy 를 최적화한다. retrieval hit rate 만 높이면 불필요한 기억을 모두 보존할 수 있고, compression ratio 만 높이면 provenance 를 지울 수 있다. hard constraint, slice 별 validation, human override, action audit 를 reward 바깥에 둔다.

Preference learning 에서 memory policy 까지

Language model post-training 에서 RLHF 가 널리 알려졌기 때문에 “sleep training 도 RLHF 인가” 라는 혼동이 생긴다. RLHF 는 가능한 training recipe 하나이고, sleep 은 언제 어떤 persistent state 를 변환할지를 규정하는 lifecycle 이다. 이 장은 preference data 가 어떻게 loss 가 되고, 그 도구를 memory controller 에 어떻게 제한적으로 재사용할 수 있는지 설명한다.

12.1 Preference pair 와 reward model

같은 prompt x 에 대한 두 response y^+, y^- 중 사람이 y^+ 를 선호하면 pairwise data (x, y^+, y^-) 가 된다. 보상 모델 $r_\psi(x, y)$ 는 보통

$$\mathcal{L}_{RM} = -\log \sigma(r_\psi(x, y^+) - r_\psi(x, y^-)) \quad (12.1)$$

을 줄여 선호 순서를 학습한다. 그 뒤 language-model policy 가 reward 를 높이도록 갱신된다. InstructGPT pipeline 은 demonstration 기반 supervised fine-tuning, comparison 기반 reward model, PPO 기반 RL 의 단계를 분리했다 (Ouyang et al. 2022).

RLHF 의 핵심 비용은 preference collection 만이 아니다. policy rollout, reward inference, reference policy 와의 KL constraint, value estimation, repeated validation 이 필요하다. 사용자별 nightly memory update 에 이 전체 pipeline 을 복제하면 비용과 instability 가 utility 를 넘을 수 있다.

RLAIF 는 AI feedback 을 사용해 label throughput 을 높인다. Constitutional AI 는 principle 에 따른 self-critique 와 AI preference 를 활용한 접근을 제시했다 (Bai et al. 2022). 그러나 teacher bias 가 자동 확장되므로, memory fact 의 truth 를 teacher preference 와 동일시하면 안 된다. AI 는 summary quality 나 scope violation 을 제안할 수 있지만 primary evidence 와 user consent 를 대체하지 않는다.

12.2 DPO: RL rollout 없이 preference loss 를 쓰는 선택지

DPO 는 preference objective 를 policy 와 reference policy 의 log-probability 차이로 직접 표현해 별도 reward model 과 online policy rollout 을 생략한다 (Rafailov et al. 2023).

$$\mathcal{L}_{DPO} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y^+|x)}{\pi_{ref}(y^+|x)} - \log \frac{\pi_\theta(y^-|x)}{\pi_{ref}(y^-|x)} \right] \right). \quad (12.2)$$

이는 training 을 단순화하지만 pair 가 나타내는 선호의 품질, reference drift, distribution coverage 문제는 남는다. Sleep-time 에서 “이 summary 가 저 summary 보다 낫다” 는 pair 에는 잘 맞을 수 있지만, long-term utility 와 capacity effect 까지 자동으로 학습되는 것은 아니다.

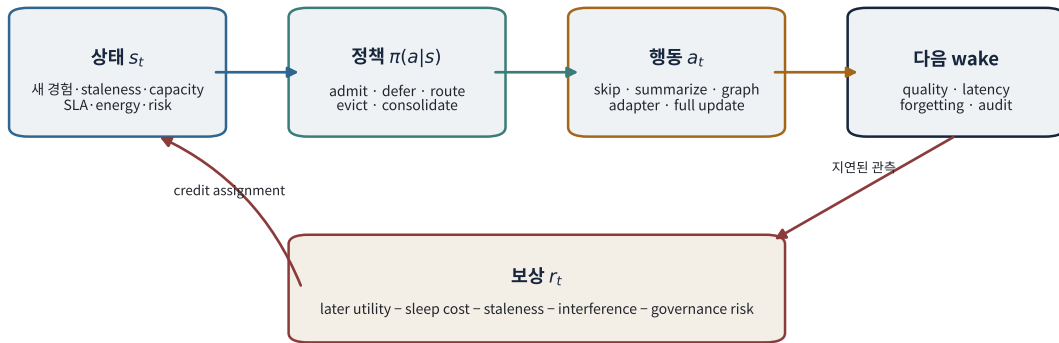
12.3 Memory controller 가 고르는 여섯 종류의 행동

기억 라우팅은 다음 결정을 하나의 policy surface 로 본다.

1. **trigger**: 지금 sleep job 을 시작할지 기다릴지,
2. **admission**: 새 event 를 버릴지 어느 tier 에 넣을지,
3. **representation**: text, vector, graph, adapter 중 무엇으로 바꿀지,
4. **replay**: 어떤 old/new example 을 어떤 비율로 읽을지,
5. **eviction**: 어떤 item/module 을 삭제 · 압축 · demote 할지,
6. **publication**: 후보를 reject, shadow, canary, promote 중 어디로 보낼지.

이 action space 는 크고 일부 action 은 되돌리기 어렵다. hierarchical controller 가 상위에서 budget 과 destination 을 고르고 하위 rule 이 item selection 을 수행하면 audit 와 안전 제약을 넣기 쉽다.

Sleep controller를 정책으로 보면



핵심 난점: 보상은 늦고, counterfactual은 관측되지 않으며, online 탐색은 사용자 상태를 손상할 수 있다.

Figure 12.1: Sleep controller 의 강화학습 loop. 실제 보상은 다음 wake period 의 utility·forgetting·비용에 서지연되어 관찰되므로 artifact lineage 와 안전한 off-policy evaluation 이 필요하다.

출처/제작: Original synthesis; SRC-STC-0051, SRC-STC-0021 대체 설명: 시스템 상태가 정책으로 들어가고 sleep 행동이 실행된 뒤 다음 wake 결과와 지연 보상이 다시 정책 학습으로 돌아오는 순환도.

12.4 Offline evaluation 이 어려운 이유

오프폴리시 평가는 후보 policy 를 실제 배포하지 않고 기존 log 로 평가한다. 행동 policy μ 가 만든 trajectory 를 target policy π 에 재가중하는 importance sampling 의 기본 형태는

$$\hat{V}(\pi) = \frac{1}{N} \sum_{i=1}^N \left(\prod_t \frac{\pi(a_t^i | s_t^i)}{\mu(a_t^i | s_t^i)} \right) G_i \quad (12.3)$$

지만 긴 horizon 에서는 weight variance 가 폭발하고, $\mu(a|s) = 0$ 인 action 은 평가할 수 없다. Sleep action 의 downstream utility 는 delayed 하고 user query 가 sparse 하므로 model-based simulator 와 doubly robust estimator 를 쓰더라도 uncertainty interval 을 보고해야 한다.

가장 강한 초기 baseline 은 learned RL policy 가 아니라 explicit rule + constrained optimizer 일 수 있다. RL 은 충분한 trajectory coverage 와 stable delayed label 이 생긴 뒤, trigger 와 budget allocation 처럼 counterfactual 을 시뮬레이션할 수 있는 좁은 action 부터 적용한다.

Table 12.1: Memory-policy reward 를 설계할 때 분리할 signal

Signal	관측 예	잘못 단독 최적화했을 때
utility	task success, correction 감소	benchmark memorization
retention	old slice 성능, calibration	새 변화에 둔감
latency	TTFT/ITL, retrieval hops	품질이 낮은 aggressive deletion
resource	GPU-hour, HBM/SSD bytes, energy	expensive rare capability 제거
governance	scope leak, delete SLA, provenance	지나친 보수성 또는 미측정 위험
operability	rollback time, version count	짧은 기간 metric 만 선호

Meta-learning, fast weights, test-time training

Pre-training 과 post-training 은 “배포 전에 무엇을 배울까” 를 주로 다룬다. Test-time training 과 sleep-time training 은 배포 뒤 상태 변화라는 공통점이 있지만 latency path, data window, validation, persistence 가 다르다. 이를 구분하려면 update 의 시간척도와 state lifetime 을 함께 본다.

13.1 Learning to learn: 초기값과 update rule 을 학습한다

메타학습은 여러 task 를 경험하며 새 task 에 적은 data 와 step 으로 잘 적응하는 초기값 또는 학습 규칙을 찾는다. MAML 의 대표적 형태는 task $\mathcal{T}_i$ 마다 내부 update

$$\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{\mathcal{T}_i}^{train}(\theta) \quad (13.1)$$

를 수행한 뒤, 그 결과의 validation loss 로 초기값 θ 를 갱신한다 (Finn et al. 2017).

이중수준 최적화는 “inner loop 에서 adaptation, outer loop 에서 그 adaptation 가능성을 평가” 하는 구조다.

$$\min_{\theta} \mathbb{E}_{\mathcal{T}} \left[\mathcal{L}_{\mathcal{T}}^{val} (U(\theta, D_{\mathcal{T}}^{train})) \right]. \quad (13.2)$$

Sleep system 에 적용하면 pre/post-training 이 좋은 consolidation initializer 와 update rule 을 배우고, 각 user/device sleep 이 local inner loop 를 실행할 수 있다. 다만 meta-training task distribution 과 실제 user stream 이 다르면 “빠르게 잘못 적응” 할 수 있으므로 outer validation 에 scope, forgetting, cost 를 포함한다.

13.2 Fast weights: 상태의 수명을 설계한다

빠른 가중치는 base weights 보다 자주 갱신하고 쉽게 reset 하는 parametric state 다. Attention KV cache, recurrent state 처럼 gradient 없이 쓰이는 state 와 혼동하지 않는다. 여기서는 learned update 로 바뀌며 forward behavior 에 들어가는 adapter, memory MLP, normalization parameter 등을 포함한다.

상태를 lifetime 으로 나누면 다음과 같다.

상태가 오래 살수록 더 많은 validation, provenance, rollback 이 필요하다. 빠른 update 가 항상 작은 compute 라는 뜻도 아니다. token 마다 backward 하면 작은 parameter 라도 latency 와 activation memory 가 커질 수 있다.

13.3 Test-time training 의 정확한 경계

테스트 시점 학습은 test input 자체 또는 가까운 stream 에서 자기지도 objective 를 만들고 parameter 를 갱신해 distribution shift 에 적응한다. 초기 연구는 한 unlabeled test sample 의 self-supervised

Table 13.1: 배포 후 학습 상태의 시간척도

상태	update 시점	persistence	실패 시 복구
query-local	한 query 내부	즉시 폐기	query 재실행
session fast state	token/session 중	session 또는 짧은 TTL	session reset
user adapter	session 사이 sleep	versioned 장기	adapter rollback
global delta	여러 cohort 통합 후	release 단위	global canary rollback
base weights	드문 offline release	장기 기준선	이전 model release

loss 로 prediction 전에 update 하는 구성을 보였다 (Sun et al. 2020); entropy minimization 으로 normalization state 를 조정하는 test-time adaptation 도 제안되었다 (Wang et al. 2021).

다음 세 가지를 구분한다.

1. **TTT-before-answer**: 현재 answer 전에 update 가 끝나므로 latency-critical 하고 현재 query 에 직접 이익이 있어야 한다.
2. **wake consolidation**: session 중 비동기 또는 짧은 idle gap 에 최근 stream 을 반영한다. 다음 query 부터 유효하지만 아직 wake resource 와 충돌한다.
3. **sleep training**: 명시적 deferred window 에서 더 넓은 replay, validation, compaction, publication 을 수행한다. 현재 query latency 에서 분리된다.

같은 LoRA update 도 첫 번째면 TTT, 세 번째면 sleep-time training 이다. 따라서 분류 기준은 optimizer 나 architecture 가 아니라 critical path 와 data horizon, persistence, release boundary 다.

13.4 Reset, accumulation, leakage

TTT 가 sample 마다 reset 되면 장기기억이 아니고, stream 에 누적되면 continual learning 이 된다. 누적형은 order sensitivity, adversarial poisoning, user cross-contamination 을 받는다. update scope 를 query/session/user/cohort/global 로 tag 하고 상위 scope 로의 승격은 sleep validation 을 거치게 한다.

Test input 을 학습에 사용한 뒤 같은 sample 에서 score 를 측정하면 transductive setting 이다. 이것을 일반 held-out generalization 과 혼동하지 않는다. 더구나 개인 memory 에서는 답을 이미 포함한 tool result 로 update 한 뒤 동일 query 를 재평가하면 retrieval cache 와 learning gain 을 분리할 수 없다.

핵심 Wake 와 sleep 은 경쟁하는 이름이 아니다

Wake/test-time update 는 즉시 adaptation 을, sleep 은 replay·compression·cross-session validation 과 long-lived publication 을 맡는다. 실제 system 은 빠른 state 가 evidence 를 모으고 sleep 이 일부를 외부장기기억이나 parametric artifact 로 승격하는 cascade 가 된다.

Model editing 과 unlearning: 좁게 바꾸고 정확히 되돌리기

Fine-tuning 은 dataset 분포 전체에 맞춰 model behavior 를 바꾸지만, 때로는 “A 의 CEO 는 B 가 아니라 C 다” 처럼 몇 개 fact 만 빨리 수정해야 한다. 반대로 삭제 요청은 특정 data 의 영향을 제거해야 한다. 두 문제 모두 update 범위, 부작용, provenance, rollback 을 요구하지만 성공 기준은 다르다.

14.1 Model editing 의 네 평가축

모델 편집은 특정 edit request $e = (x_e, y_e)$ 를 반영하면서 나머지 behavior 를 보존하려는 수정이다. ROME 은 transformer MLP 의 factual association 을 찾아 rank-one update 를 적용하고 (Meng et al. 2022a), MEMIT 은 이를 많은 memory edit 로 확장했다 (Meng et al. 2022b). 이들은 “fact 가 한 주소에 완전히 저장된다” 는 증명이 아니라, 특정 benchmark 와 architecture 에서 효과적인 intervention 을 보인 방법이다.

평가는 최소 네 축으로 나눈다.

1. **efficacy**: edit prompt 에서 새 답을 내는가?
2. **paraphrase/generalization**: 표현이 달라도 같은 correction 을 쓰는가?
3. **specificity**: 비슷하지만 다른 entity-relation 은 보존되는가?
4. **retention**: unrelated capability 와 이전 edit 가 유지되는가?

국소성은 세 번째와 네 번째를 포괄한다. 단순히 edit prompt log-probability 만 높이면 model 이 주변 사실까지 바꾸거나 같은 token 을 과도하게 선호할 수 있다. Sleep batch 에는 edit example 뿐 아니라 neighborhood, counterfactual, old capability probe 가 필요하다.

14.2 많은 edit 는 다시 continual learning 과 capacity 문제가 된다

한두 edit 는 저랭크 update 로 가능해 보여도 수천 · 수백만 edit 를 누적하면 interference 가 늘어난다. 같은 subject 의 시간별 fact 가 충돌하고, 이전 edit 가 새 edit 에 의해 덮이며, 어느 update 를 제거해야 하는지 모호해진다. 매일 editing 을 반복하는 시스템은 결국 다음 중 하나를 선택해야 한다.

- latest fact 만 weights 에 남기고 history 는 temporal external memory 에 둔다.
- edit 를 versioned adapter 에 격리하고 주기적으로 merge/distill 한다.
- 반복 조희되고 안정적인 fact 만 parametric promotion 하고 나머지는 retrieval 한다.
- global knowledge refresh 는 큰 offline model release 에 맡긴다.

즉 editing 은 sleep-time compute 의 가능한 operator 이지만, 무한 append 가능한 장기 기억 자체는 아니다.

14.3 Unlearning 은 delete key 보다 강한 요구다

머신 언러닝은 지정 data 가 학습에 없었던 counterfactual model 에 가까워지도록 그 영향을 제거하는 절차다. Sharded, Isolated, Sliced, Aggregated (SISA) training 은 data shard 별 model 을 분리해 삭제 시 영향받은 부분만 재학습하는 방향을 제시했다 (Bourtole et al. 2021).

External memory 는 item pointer 를 지우기 쉬워도 cache, embedding replica, graph edge, summary, downstream training dataset 까지 삭제가 전파되어야 한다. Parametric memory 는 어느 weight 가 한 record 에 기여했는지 역함수가 없으므로 exact deletion 이 훨씬 어렵다. “답을 더 이상 말하지 않는다” 와 “data 영향이 제거되었다” 도 다르다.

Table 14.1: 수정 · 삭제 destination 별 복구 성질

Destination	수정 단위	삭제/롤백	잔여 위험
raw text/log	record/segment	pointer+replica 삭제	파생 summary-dataset
vector index	vector+metadata	tombstone+rebuild	cache 와 stale replica
knowledge graph	node/edge/time interval	edge version rollback	derived edge 와 conflict
adapter/LoRA	file 또는 rank component	version detach	merged artifact, shared behavior
base weights	diffuse parameters	retrain/edit/unlearn	비국소 영향, 검증 비용

14.4 Provenance graph 가 architecture choice 를 바꾼다

권리 만료가 잦거나 user delete SLA 가 짧다면 external memory 나 isolated adapter 의 운영 가치가 커진다. Stable public knowledge 처럼 많은 query 가 반복되고 source set 이 고정될 때는 global parametric refresh 가 유리할 수 있다. 따라서 parametric/external 선택은 accuracy 비교만이 아니라 expected delete frequency, scope, edit conflict, rollback time 을 포함한 lifecycle cost 비교다.

주의 Negative answer 만으로 unlearning 을 증명하지 말 것

모델이 특정 prompt 에 답하지 않도록 fine-tune 한 것은 suppression 일 수 있다. membership inference, paraphrase, related representation, downstream model lineage 를 포함해 요구 수준을 정의해야 한다. 완전한 counterfactual retraining 과 근사 unlearning 을 보고서에서 구분한다.

Training systems: HBM, 통신, checkpoint, publication

Sleep-time compute 를 “남는 GPU 에서 background fine-tuning” 으로만 보면 실제 병목을 놓친다. Training 은 weights 외에 gradients, optimizer states, activations, batches 를 동시에 움직이고, 후보 artifact 를 검증 · 복제 · 배포해야 끝난다. 이 장은 알고리즘을 시스템 자원으로 번역한다.

15.1 한 step 의 memory accounting

Parameter 수를 P , element byte 를 b_w, b_g, b_o 라 하면 단순한 state lower bound 는

$$M_{state} \approx Pb_w + Pb_g + kPb_o + M_{act} + M_{temp}, \quad (15.1)$$

이다. Adam 의 moment 가 두 개면 $k = 2$ 이고, master weight 를 별도 유지하면 한 항이 더해진다. Inference 는 gradient 와 optimizer state 가 없지만 training 은 forward activation 을 backward 까지 보존한다.

혼합 정밀도는 matrix multiply 와 저장 일부를 FP16/BF16/FP8 로 낮추고 필요한 accumulation 이나 master state 는 더 높은 precision 에 두어 throughput 과 안정성을 trade-off 한다 (Micikevicius et al. 2018). “모든 것을 16-bit” 라는 뜻이 아니므로 model state 별 dtype 을 workbook 과 checkpoint manifest 에 적는다.

활성 체크포인트링은 모든 intermediate activation 을 저장하지 않고 일부 boundary 만 보존한 뒤 backward 에서 forward 일부를 재계산한다. 깊이 n network 의 memory 를 sublinear 하게 줄이는 대신 추가 compute 를 쓰는 trade-off 가 제시되었다 (Chen et al. 2016). Sleep cluster 에 idle FLOPs 가 있어도 재계산이 wall-clock deadline 과 energy budget 을 넘는지 봐야 한다.

15.2 병렬화는 무엇을 복제하고 무엇을 통신하는가

데이터 병렬화는 각 device 에 model replica 를 두고 다른 batch shard 를 처리한 뒤 gradient 를 all-reduce 한다. Compute 는 잘 확장하지만 weights 와 optimizer state 가 복제되고, 짧은 per-user sleep job 은 batch 가 작아 device utilization 이 낮을 수 있다.

텐서 병렬화는 한 layer 의 matrix 를 여러 device 에 나눈다. 큰 model 을 fit 할 수 있지만 layer 마다 collective communication 이 발생한다. Megatron-LM 은 transformer 내부 model parallel pattern 을 체계화했다 (Shoeybi et al. 2019).

파이프라인 병렬화는 layer block 을 stage 로 나누고 microbatch 를 흐르게한다. GPipe 는 large model training 을 위한 pipeline schedule 을 제시했다 (Huang et al. 2019). Microbatch 가 적으면 stage bubble 이 커지므로 작은 sleep job 을 개별 pipeline 에 넣기보다 여러 tenant job 을 privacy boundary 안에서 묶는 scheduler 가 필요하다.

샤딩은 parameter, gradient, optimizer state 를 device 에 나눠 중복을 줄인다. ZeRO 는 data-parallel training 의 state redundancy 를 단계적으로 제거한다 (Rajbhandari et al. 2020). 그러나 shard 를 NVMe 나 remote memory 까지 offload 하면 HBM capacity 는 줄어도 step 마다 read/write traffic 과 tail latency 가 늘 수 있다.

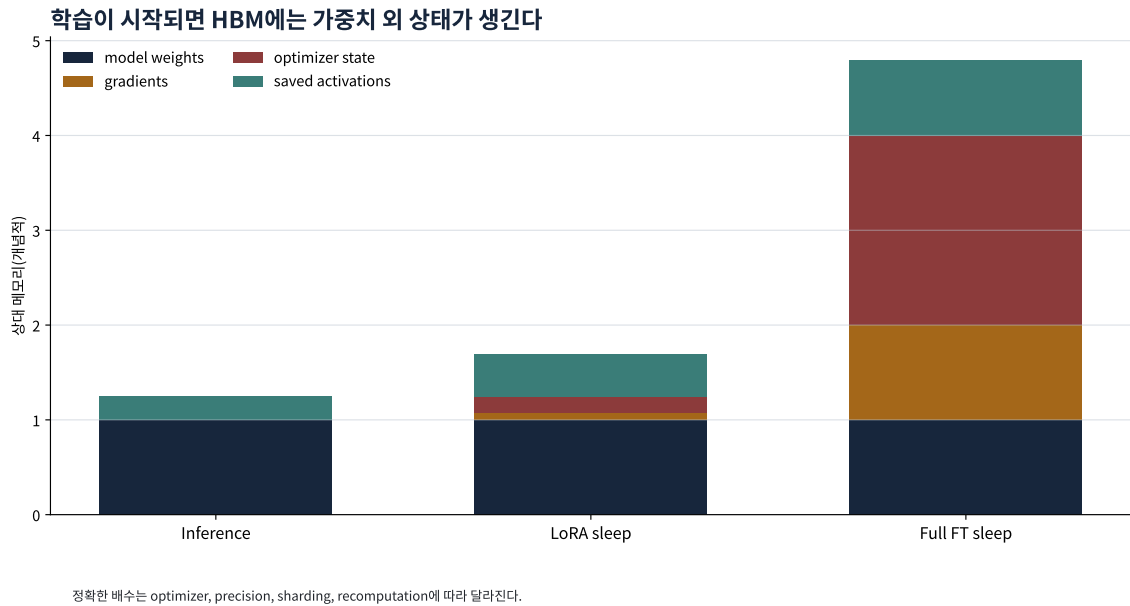


Figure 15.1: Inference, LoRA sleep, full fine-tuning sleep 의 상대적 state stack. 실제 값은 optimizer, precision, sequence, checkpointing, sharding 에 따라 달라지므로 이 그림은 구성요소 지도다.

출처/제작: Original synthesis; SRC-STC-0030, SRC-STC-0021 대체 설명: 세 가지 workload 마다 weights, gradients, optimizer state, activations 가 층으로 쌓이며 full fine-tuning 이 가장 많은 상태를 갖는 비교 그림.

Table 15.1: Sleep training job 의 주요 data movement

경로	이동하는 상태	최적화 질문
storage → host	replay examples, provenance	sequential batch, compression, filtering locality
host → HBM	tokens, weights, optimizer shard	pin/prefetch, tenant isolation, reuse
GPU ↔ GPU	activations, gradients, parameters	topology-aware collective, job packing
HBM ↔ NVMe/CXL	cold optimizer, checkpoint, adapter	bandwidth vs recompute, endurance
candidate → serving	weights/delta/index/graph	atomic publish, cache warmup, roll-back copy

15.3 Checkpoint 가 아니라 release bundle 을 version 한다

버전 관리는 model file 이름만 바꾸는 일이 아니다. 하나의 candidate version 은 base model hash, adapter/delta, optimizer continuation state, dataset manifest, code/container, random seed, hyperparameters, evaluation report, source freeze 를 묶는다. 그래야 동일 candidate 를 재현하거나 어떤 source 가 들어갔는지 판단할 수 있다.

롤백은 새 artifact pointer 를 이전 검증 version 으로 원자적으로 전환하고, cache 와 replica 를 같은 epoch 으로 맞추며, 실패 candidate 로 만든 후속 data 를 quarantine 하는 절차다. Rollback time objective(RTO) 와 최대 허용 data loss(RPO) 를 external index, adapter, global weights 각각 정의한다.

출판 게이트는 candidate 가 offline metric 뿐 아니라 retention, privacy scope, delete compliance, latency, resource budget 을 통과할 때만 serving state 로 승격하는 boundary 다. Training 완료와 publication 완료를 분리하면 실패 후보를 안전하게 분석할 수 있다.

관측 가능성은 sleep job 하나를 event ID 에서 serving effect 까지 trace 할 수 있는성질이다. 최소 metric 은 queue wait, data bytes, H2D/D2H bytes, FLOPs, HBM peak, checkpoint bytes, validation slices, promotion result, next-wake utility, rollback event 다. 평균만이 아니라 tenant 와 job size 별 tail 을 본다.

15.4 Memory device 관점의 기회

Sleep workload 는 latency-insensitive 하다고 해서 bandwidth-insensitive 하지 않다. Full fine-tuning 은 optimizer·gradient 의 높은-capacity state, replay 는 sequential/filtered reads, checkpoint 는 large burst writes, 외부 기억 compaction 은 metadata-rich random I/O 를 만든다. 따라서 device solution 은 다음 primitive 를 목표로 할 수 있다.

- HBM: active weights, activation, hot optimizer shard 의 bandwidth 와 capacity,
- CXL/host memory: cold optimizer·multi-tenant adapter 의 coherent tiering,
- NVMe: versioned checkpoint, replay log, index rebuild 의 endurance 와 predictable QoS,
- near-data processing: dedup, compression, vector/graph compaction, checksum/provenance scan,
- secure memory: tenant key, erase proof, immutable lineage metadata.

기회는 broad parametric sleep 이 주류가 되지 않아도 남는다. External-memory consolidation, global model refresh, checkpointing, index compaction 역시 같은 data movement 와 versioning primitive 를 사용하기 때문이다.

핵심 Roofline 만으로 training job 을 설명하지 못한다

Kernel compute/HBM roofline 외에 interconnect collective, host/storage ingest, checkpoint write, validation inference, artifact replication 을 end-to-end critical path 에 넣는다. Sleep window deadline 을 넘기면 다음 wake 전에 publication 하지 못하므로 throughput 이 곧 utility 다.

End-to-end mapping: wake 경험에서 검증된 장기기억까지

앞 장의 용어를 하나의 lifecycle 로 조립한다. **Wake** 상태는 사용자 요청을 처리하는 critical path 에서 읽히고, query/session 범위에서 빠르게 변할 수 있다. **Sleep** 상태는 응답 path 밖에서 replay·학습·압축·검증되는 후보다. “밤”이라는 시각보다 dependency 와 latency boundary 가 정의의 핵심이다.

16.1 두 기억 destination 과 hybrid promotion

외부 기억은 text, vector, graph, log 처럼 model 밖에서 provenance 와 함께관리되고 query 때 검색된다. 모수적 기억은 base weights, LoRA, adapter, fast weights 처럼 forward 함수 안에서 behavior 를 바꾼다.

External memory 는 쓰기·삭제·scope-source attribution 이 쉽고 capacity 를 독립 확장할 수 있지만 retrieval latency, context token, index drift, tool failure 가 있다. Parametric memory 는 retrieval 없이 behavior 와 skill 을 바꿀 수 있지만 update·검증·rollback 이 비싸고 exact provenance 와 deletion 이 어렵다. 따라서 실용적 default 는 external system-of-record 에 먼저 저장하고, 반복 사용·안정성·scope-break-even 이 충분한 pattern 만 reversible parametric artifact 로 승격하는 hybrid 다.

16.2 모든 sleep operator 를 같은 schema 로 기록한다

Table 16.1: Sleep operator 의 공통 training/serving schema

Operator	입력 data	변경 state	objective/signal	validation	publication/rollback
summary	episode + source	text record	fidelity+compression	QA, temporal, provenance	record version/tombstone
re-embed	text+encoder	vector index	retrieval utility	recall, scope, latency	index epoch/rebuild
graph merge	facts+time	node/edge	consistency+coverage	conflict, traversal	edge version/replay log
replay FT	new+coreset	LoRA/adapter	task+retention loss	new/old/safety slices	detach previous adapter
distill	teacher outputs	student/delta	KL+task loss	fidelity, utility, tail	teacher+student pair
model edit	edit+neighbors	localized weights	efficacy+locality	paraphrase, specificity	inverse/version rollback
global refresh	curated corpus	base weights	multi-objective	full release suite	model release rollback

각 row 는 반드시 teacher/source, trainable state, loss/reward, optimizer, capacity destination, validation, version 을 갖는다. Training kernel 이 없는 summary/graph merge 도 data transformation objective 와 evaluation, publication 이 있으므로 넓은 의미의 sleep compute 에 포함된다. 반대로 아무 validation 없이 log 를 append 하는 것은 consolidation 이 아니라 storage 다.

16.3 Worked example: 반복되는 사용자 correction

사용자가 여러 session 에서 “회의 요약은 표보다 세 문장 bullet 을 선호한다” 고 수정했다고 하자.

1. **Wake capture:** correction event 와 이전 response, explicit user scope, timestamp 를 append 한다.
2. **Curation:** 일시적 요청인지 stable preference 인지 분류하고 중복을 cluster 한다.
3. **External update:** user profile text/graph 에 source-linked preference 를 versioned write 한다.
4. **Benefit measurement:** retrieval 이 실제 prompt token 과 latency 를 얼마나 늘리고, preference 가 얼마나 자주 재사용되는지 관찰한다.
5. **Candidate promotion:** 반복 빈도와 stable period 가 threshold 를 넘으면 user-scoped LoRA training set 에 positive/negative pair 와 general retention replay 를 넣는다.
6. **Validation:** preference success, unrelated style, safety, other-user non-leakage, latency 를 비교한다.
7. **Publish:** external profile 은 system-of-record 로 유지하고 adapter pointer 만 canary 승격 한다.
8. **Evict/rollback:** preference 가 바뀌면 old external record 를 supersede 하고 adapter 를 detach 또는새 version 으로 대체한다.

이 예에서 external 과 parametric memory 는 경쟁 복제품이 아니다. 하나는 evidence 와 control plane, 다른 하나는 반복되는 behavior 의 serving accelerator 다.

16.4 Break-even 을 묻는 최소 식

Parametric promotion 의 one-time sleep cost 를 $C_{train} + C_{validate} + C_{publish}$, query 당 retrieval 에서 절약되는 기대 비용을 ΔC_{wake} , 예상 유효 query 수를 N , 추가 risk 와 maintenance 를 $C_{risk} + C_{maint}$ 라 하면 promotion 조건의 한 baseline 은

$$N \Delta C_{wake} > C_{train} + C_{validate} + C_{publish} + C_{risk} + C_{maint}. \quad (16.1)$$

정확도/utility gain 도 통화 또는 constrained objective 로 별도 포함한다. 이 식은 scaling law 가 아니라측정 항목을 빠뜨리지 않기 위한 accounting identity 다. N 을 과대 추정하거나 validation cost 를 0 으로두면 대부분의 기억을 weights 로 보내는 잘못된 결론이 나온다.

16.5 운영 decision tree

1. source 와 scope 가 불명확하면 학습하지 말고 quarantine 한다.
2. exact delete, temporal history, citation 이 중요하면 external system-of-record 를 유지한다.
3. current query 만 적응하면 query/session-local state 로 제한하고 reset 한다.
4. 반복 사용되는 stable pattern 이며 retrieval overhead 가 크면 reversible adapter 후보를 만든다.
5. 여러 사용자에게 일반화되고 독립 evidence 가 충분하면 global refresh queue 로 승격한다.
6. 어느 경우든 capacity action 과 rollback artifact 가 없으면 publish 하지 않는다.

핵심 Study Paper 를 읽을 때의 질문

새 sleep-time 연구를 볼 때 “성능이 올랐나” 에 앞서 다음을 확인한다. Wake/sleep 경계는 무엇인가? 무슨 state 가 얼마나 오래 바뀌는가? Data source 와 replay 는 무엇인가? Old capability 를 어떻게 평가했는가? Capacity 가 찼을 때 무엇을 지우는가? Candidate 는 어떻게 publish·rollback 되는가? 이 여섯 질문이 논문결과를 실제 system 과 memory-device workload 로 번역한다.

용어 사전과 개념 의존성

이 사전은 Study Paper 가 참조하는 canonical concept registry 에서 자동 생성한다. 용어를 단독 정의로만 읽지 말고, 세 번째 열의 prerequisite 를 먼저 확인한다. 본문의 파란 링크는 각 개념이 처음 설명되는 절로 이동한다.

Table G.1: Sleep-time compute 를 읽기 위한 canonical glossary

용어 (concept ID)	한 문장 정의	선행 개념
데이터 data	학습이나 평가에 사용되는 관측, 예시, 선호, 경험의 집합이다.	없음
파라미터 parameter	학습으로 값이 바뀌며 모델의 함수를 결정하는 수치다.	없음
순전파 forward-pass	입력과 현재 파라미터로 예측을 계산하는 과정이다.	data, parameter
목적함수 objective	학습이 줄이거나 늘리려는 전체 수치화 목표다.	forward-pass
손실 loss	한 예측 또는 배치가 목표와 얼마나 어긋났는지 나타내는 학습 신호다.	objective
기울기 gradient	각 파라미터를 조금 바꿀 때 손실이 어느 방향으로 얼마나 변하는지 나타낸다.	loss, parameter
역전파 backpropagation	출력의 손실에서 각 파라미터의 기울기를 연쇄법칙으로 계산하는 절차다.	forward-pass, gradient
옵티마이저 optimizer	기울기와 내부 상태를 사용해 파라미터를 갱신하는 규칙이다.	gradient
학습률 learning-rate	한 갱신에서 기울기를 얼마나 크게 반영할지 정하는 크기다.	optimizer
옵티마이저 상태 optimizer-state	모멘텀과 분산 추정치처럼 옵티마이저가 파라미터 외에 유지하는 상태다.	optimizer
배치 batch	한 번의 갱신을 위해 함께 처리하는 예시 묶음이다.	data, loss
에폭 epoch	학습 데이터 전체를 대략 한 번 소비한 단위다.	batch
샘플링 sampling	전체 데이터에서 다음 학습 예시나 배치를 선택하는 규칙이다.	data, batch
검증 세트 validation-set	학습에는 직접 쓰지 않고 설정 선택과 조기 종료에 사용하는 데이터다.	data

다음 쪽에 계속

용어 (concept ID)	한 문장 정의	선행 개념
테스트 세트 test-set	최종 선택 뒤 일반화 성능을 추정하기 위해 보존하는 데이터다.	validation-set
과적합 overfitting	학습 데이터에는 잘 맞지만 새로운 데이터에는 성능이 낮아지는 현상이다.	validation-set
일반화 generalization	학습에서 보지 못한 입력에서도 유용한 성능을 유지하는 능력이다.	test-set, overfitting
사전학습 pretraining	배포 전 대규모 데이터로 범용 모델을 처음 형성하는 학습이다.	objective, batch
지속 사전학습 continued-pretraining	기존 사전학습 모델을 추가 비지도 또는 자기지도 코퍼스로 계속 학습하는 방식이다.	pretraining
미세조정 fine-tuning	사전학습 모델을 더 좁은 데이터와 목적에 맞게 추가 학습하는 방식이다.	pretraining
지시 튜닝 instruction-tuning	지시와 응답 쌍으로 지시 따르기 행동을 학습하는 미세조정이다.	fine-tuning, supervision
선호 최적화 preference-optimization	사람 또는 모델이 선호한 출력 쪽으로 정책을 이동시키는 후학습 계열이다.	fine-tuning, loss
지도 신호 supervision	입력에 대해 원하는 목표나 비교 신호를 외부에서 제공하는 학습 방식이다.	data, loss
자기지도학습 self-supervision	원시 데이터 자체에서 마스킹이나 다음 토큰 같은 목표를 구성하는 학습 방식이다.	supervision
표현 representation	모델이 입력의 정보를 후속 계산에 쓰기 좋은 내부 벡터나 상태로 표현한 것이다.	forward-pass
로짓 logit	확률 정규화 전 모델이 각 선택지에 부여한 점수다.	forward-pass
소프트맥스 softmax	로짓을 합이 1 인 확률 분포로 바꾸는 함수다.	logit
교차엔트로피 cross-entropy	목표 분포에 높은 확률을 주도록 만드는 대표적 분류 · 언어 모델 손실이다.	softmax, loss
지식 증류 distillation	교사 모델의 출력이나 내부 신호를 학생 모델이 모방하도록 학습하는 방식이다.	loss, softmax
교사 모델 teacher-model	증류 목표를 제공하는 기준 모델 또는 모델 집합이다.	distillation
학생 모델 student-model	교사의 행동이나 분포를 학습하는 대상 모델이다.	distillation
KL 발산 kl-divergence	두 확률 분포의 불일치를 한 방향으로 측정하는 양이다.	softmax

다음 쪽에 계속

용어 (concept ID)	한 문장 정의	선행 개념
온도 temperature	확률 분포의 날카로움과 탐색 정도를 조절하는 스케일이다.	softmax
PEFT peft	전체 가중치 대신 작은 일부 상태만 학습하는 파라미터 효율 미세조정이다.	fine-tuning, parameter
LoRA lora	가중치 변화량을 두 저랭크 행렬의 곱으로 제한하는 PEFT 방식이다.	peft
어댑터 adapter	기본 모델 사이에 삽입하거나 별도로 결합하는 작은 학습 모듈이다.	peft
MoE mixture-of-experts	입력마다 일부 전문가 모듈만 선택해 계산하는 희소 모델 구조다.	parameter, forward-pass
학습 가능 상태 trainable-state	한 학습 단계에서 실제로 갱신 가능한 파라미터와 보조 상태의 집합이다.	parameter, optimizer-state
서빙 산출물 serving-artifact	학습이 끝난 뒤 추론에 배포되는 가중치, 어댑터, 인덱스 또는 그래프다.	trainable-state
데이터 큐레이션 data-curation	목표와 정책에 맞는 학습 후보를 수집 · 정규화 · 중복 제거 · 분류하는 과정이다.	data
데이터 필터링 data-filtering	품질 · 안전 · 권리 · 관련성 기준으로 학습 후보를 남기거나 제외하는 절차다.	data-curation
데이터 증강 data-augmentation	의미상 보존하려는 속성을 유지하면서 학습 예시의 변형을 만드는 방법이다.	data
합성 데이터 synthetic-data	모델 · 규칙 · 시뮬레이터가 생성한 학습 또는 평가 예시다.	data
출처 추적성 provenance	데이터와 산출물이 어디서 왔고 어떤 변환 · 검증을 거쳤는지 추적하는 기록이다.	data
리플레이 버퍼 replay-buffer	미래 학습에서 다시 사용할 과거 경험이나 예시를 제한된 용량으로 보관한 저장소다.	sampling, data
코어셋 coreset	전체 과거 데이터를 대표하도록 선택한 작은 가중 표본 집합이다.	replay-buffer
생성 리플레이 generated-replay	원본 예시 대신 생성 모델이 복원한 과거 분포의 표본을 재학습에 쓰는 방식이다.	synthetic-data, replay-buffer
지속학습 continual-learning	시간에 따라 도착하는 데이터나 과제를 배우면서 이전 능력을 유지하는 학습 설정이다.	fine-tuning
파국적 망각 catastrophic-forgetting	새 학습 뒤 이전 데이터나 과제 성능이 급격히 무너지는 현상이다.	continual-learning
안정성-가소성 stability-plasticity	기존 능력을 안정적으로 보존하는 힘과 새 정보를 빠르게 배우는 가소성 사이의 긴장이다.	continual-learning

다음 쪽에 계속

용어 (concept ID)	한 문장 정의	선행 개념
후방 전이 backward-transfer	새 학습이 과거 과제 성능을 높이거나 낮추는 효과다.	continual-learning
정규화 regularization	원하는 해의 성질이나 과거 상태 보존을 손실에 제약으로 더하는 방법이다.	loss
EWC ewc	과거 과제에 중요했던 가중치의 이동을 중요도에 비례해 벌점 주는 지속학습 방법이다.	regularization, catastrophic-forgetting
기울기 투영 gradient-projection	새 기울기를 허용된 제약 영역으로 투영해 과거 손실의 악화를 제한하는 방법이다.	gradient
GEM gem	episodic memory 의 과거 기울기를 제약으로 사용하는 Gradient Episodic Memory 방법이다.	gradient-projection, replay-buffer
파라미터 격리 parameter-isolation	과제 · 사용자 · 시기별로 다른 파라미터 부분을 배정해 간섭을 막는 방법이다.	parameter, continual-learning
동적 확장 dynamic-expansion	기존 용량이 부족할 때 모듈이나 파라미터를 동적으로 추가하는 방법이다.	parameter-isolation
프루닝 pruning	중요도가 낮은 연결 · 모듈 · 기억을 제거해 용량과 계산을 회수하는 과정이다.	parameter
용량 예산 capacity-budget	정확도 · 지연 · 비용 · 권리 제약 아래 한 계층이 사용할 수 있는 저장 · 상태 한도다.	serving-artifact
압축 compression	허용 가능한 정보 손실 아래 상태나 데이터를 더 작은 표현으로 바꾸는 과정이다.	capacity-budget
율-왜곡 rate-distortion	표현률 또는 저장 비용과 허용 왜곡 사이의 최적 trade-off 를 다루는 관점이다.	compression, loss
기억 입장 정책 memory-admission	새 경험을 어느 기억 계층에 넣을지 또는 버릴지 결정하는 정책이다.	capacity-budget
기억 퇴거 정책 memory-eviction	용량을 회수하기 위해 어떤 기존 기억을 제거 · 축약 · 하향 이동할지 정하는 정책이다.	memory-admission
공고화 consolidation	여러 경험을 재생 · 요약 · 결합해 더 안정적이고 장기적인 상태로 전환하는 과정이다.	continual-learning, replay-buffer
RL 상태 rl-state	의사결정 시점에 정책이 관측하는 환경 · 시스템 · 기억의 정보다.	data
행동 action	정책이 상태를 보고 선택해 환경이나 기억 시스템을 바꾸는 조작이다.	rl-state
보상 reward	행동 결과의 바람직함을 수치로 제공하는 학습 신호다.	action
궤적 trajectory	시간 순서로 이어진 상태 · 행동 · 보상의 열이다.	rl-state, action, reward

다음 쪽에 계속

용어 (concept ID)	한 문장 정의	선행 개념
리턴 return	현재 이후 보상을 할인해 더한 누적 성과다.	trajectory
정책 policy	상태에서 행동의 확률 또는 선택으로 가는 의사결정 규칙이다.	rl-state, action
가치 함수 value-function	한 상태나 상태-행동에서 기대되는 미래 return 을 추정하는 함수다.	return, rl-state
신용 할당 credit-assignment	지연된 결과의 책임이나 공을 과거 행동과 상태에 배분하는 문제다.	return, action
정책 기울기 policy-gradient	기대 return 이 증가하도록 정책 파라미터의 기울기를 추정하는 방법이다.	policy, gradient, return
온폴리시 on-policy	현재 학습 중인 정책이 수집한 경험으로 그 정책을 갱신하는 설정이다.	policy
오프폴리시 off-policy	현재 정책과 다른 과거 · 행동 정책이 수집한 경험도 학습에 사용하는 설정이다.	policy, replay-buffer
보상 모델 reward-model	사람이나 모델의 비교 선호에서 출력의 품질 점수를 근사한 모델이다.	preference-optimization
RLHF rlhf	사람 피드백으로 학습한 보상이나 선호를 이용해 모델 정책을 조정하는 절차다.	reward-model, policy-gradient
RLAIF rlaif	사람 대신 또는 사람과 함께 AI 가 만든 피드백을 이용하는 선호 학습 절차다.	rlhf
DPO dpo	명시적 보상 모델과 online RL rollout 없이 선호 쌍으로 정책을 직접 최적화하는 방법이다.	preference-optimization
오프폴리시 평가 off-policy-evaluation	새 정책을 실제 배포하지 않고 과거 로그로 기대 성과를 추정하는 평가다.	off-policy, validation-set
메타학습 meta-learning	새 과제나 데이터에 적은 단계로 잘 적응하도록 학습 절차 자체를 학습하는 관점이다.	fine-tuning
이중수준 최적화 bilevel-optimization	내부 적응 결과를 바깥 목표로 평가해 초기값이나 학습 규칙을 최적화하는 중첩 문제다.	meta-learning, objective
빠른 가중치 fast-weights	base weights 보다 빠르게, 짧은 범위에서 갱신 · 폐기되는 파라미터성 상태다.	parameter
테스트 시점 학습 test-time-training	배포 후 현재 또는 최근 입력에서 만든 목표로 추론 경로 안팎에서 상태를 갱신하는 학습이다.	fast-weights, self-supervision
모델 편집 model-editing	좁은 사실이나 행동을 국소적으로 바꾸면서 나머지 능력을 보존하려는 모델 수정이다.	parameter
국소성 locality	의도한 대상 밖의 예측과 능력이 수정 전과 가깝게 유지되는 성질이다.	model-editing, generalization

다음 쪽에 계속

용어 (concept ID)	한 문장 정의	선행 개념
머신 언러닝 unlearning	지정 데이터나 그 영향이 학습되지 않은 상태에 가깝도록 제거하는 절차다.	model-editing, provenance
혼합 정밀도 mixed-precision	정확도와 안정성을 지키며 연산·상태별로 서로 다른 수치 정밀도를 쓰는 학습 기법이다.	trainable-state
활성 체크포인팅 activation-checkpointing	일부 activation 만 저장하고 backward 때 나머지를 재계산해 메모리를 절약하는 기법이다.	forward-pass
데이터 병렬화 data-parallelism	장치마다 다른 batch shard 를 처리하고 gradient 를 결합하는 병렬화다.	batch
텐서 병렬화 tensor-parallelism	한 층의 텐서 연산과 파라미터를 여러 장치에 나누는 병렬화다.	parameter
파이프라인 병렬화 pipeline-parallelism	연속된 층 구간을 여러 stage 에 배치하고 microbatch 를 파이프라인으로 흘리는 병렬화다.	forward-pass
샤딩 sharding	파라미터·gradient-optimizer state 를 장치나 저장 계층에 분할 배치하는 방법이다.	optimizer-state, parameter
버전 관리 versioning	학습 데이터·코드·상태·평가·산출물을 함께 식별 가능한 버전으로 묶는 관리다.	serving-artifact
롤백 rollback	실패한 새 상태를 중단하고 검증된 이전 상태로 원자적으로 되돌리는 절차다.	versioning
출판 게이트 publication-gate	후보 상태가 검증·안전·권리·비용 기준을 통과할 때만 serving 에 승격하는 경계다.	validation-set, versioning
Wake 상태 wake-state	사용자 요청을 처리하는 latency-critical 경로에서 읽거나 짧게 갱신되는 상태다.	serving-artifact
Sleep 상태 sleep-state	응답 경로 밖에서 재생·압축·학습·검증·승격되는 deferred 상태다.	wake-state, continual-learning
모수적 기억 parametric-memory	모델 가중치·어댑터·fast weights 처럼 forward 함수 안에 내재된 기억 상태다.	parameter
외부 기억 external-memory	text·vector·graph·log 처럼 모델 밖에 저장되어 검색과 도구 호출로 사용되는 기억이다.	data
기억 라우팅 memory-routing	질의와 상태에 따라 외부 기억·fast state·parametric state 중 어디를 읽고 쓸지 정하는 제어다.	external-memory, parametric-memory
관측 가능성 observability	데이터·학습·평가·배포 상태와 실패 원인을 지표·로그·trace 로 재구성할 수 있는 성질이다.	publication-gate

핵심 의존성 표를 사용하는 법

어떤 Study claim 이 낫설다면 해당 concept ID 의 본문 label 로 이동하고, 선행 개념을 왼쪽에서 오른쪽 순서로 읽는다. 용어 정의는 결과를 보장하지 않는다. 예를 들어 unlearning 의 정의를 안다는 것은 특정 방법이 삭제 요구를 완전히 충족한다는 뜻이 아니다.

참고문헌

- Anonymous (2026). *What to Keep, What to Forget: A Rate–Distortion View of Memory Compaction in LLMs and Agents*. arXiv: 2607.08032. URL: <https://arxiv.org/abs/2607.08032v1>.
- Bai, Yuntao et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv: 2212.08073. URL: <https://arxiv.org/abs/2212.08073>.
- Bourtoutle, Lucas, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot (2021). “Machine Unlearning”. In: *2021 IEEE Symposium on Security and Privacy*, pp. 141–159. doi: 10.1109/SP40001.2021.00019.
- Chaudhry, Arslan, Marc’Aurelio Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc’Aurelio Ranzato (2019). *On Tiny Episodic Memories in Continual Learning*. arXiv: 1902.10486. URL: <https://arxiv.org/abs/1902.10486>.
- Chen, Tianqi, Bing Xu, Chiyuan Zhang, and Carlos Guestrin (2016). *Training Deep Nets with Sublinear Memory Cost*. arXiv: 1604.06174. URL: <https://arxiv.org/abs/1604.06174>.
- Finn, Chelsea, Pieter Abbeel, and Sergey Levine (2017). “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks”. In: *Proceedings of the 34th International Conference on Machine Learning*, pp. 1126–1135. URL: <https://proceedings.mlr.press/v70/finn17a.html>.
- Hu, Edward J. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv: 2106.09685. URL: <https://arxiv.org/abs/2106.09685v2>.
- Huang, Yanping et al. (2019). “GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism”. In: *Advances in Neural Information Processing Systems*. Vol. 32. URL: <https://papers.nips.cc/paper/8305-gpipe-efficient-training-of-giant-neural-networks-using-pipeline-parallelism>.
- Kirkpatrick, James et al. (2017). “Overcoming Catastrophic Forgetting in Neural Networks”. In: *Proceedings of the National Academy of Sciences* 114.13, pp. 3521–3526. doi: 10.1073/pnas.1611835114.
- Lopez-Paz, David and Marc’Aurelio Ranzato (2017). “Gradient Episodic Memory for Continual Learning”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/1706.08840>.
- McClelland, James L., Bruce L. McNaughton, and Randall C. O’Reilly (1995). “Why There Are Complementary Learning Systems in the Hippocampus and Neocortex”. In: *Psychological Review* 102.3, pp. 419–457. doi: 10.1037/0033-295X.102.3.419.
- Meng, Kevin, David Bau, Alex Andonian, and Yonatan Belinkov (2022a). *Locating and Editing Factual Associations in GPT*. arXiv: 2202.05262. URL: <https://arxiv.org/abs/2202.05262v5>.
- Meng, Kevin, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau (2022b). *Mass-Editing Memory in a Transformer*. arXiv: 2210.07229. URL: <https://arxiv.org/abs/2210.07229v2>.
- Micikevicius, Paulius et al. (2018). “Mixed Precision Training”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=r1gs9JgRZ>.
- Ouyang, Long et al. (2022). “Training Language Models to Follow Instructions with Human Feedback”. In: *Advances in Neural Information Processing Systems*. Vol. 35, pp. 27730–27744. URL: <https://arxiv.org/abs/2203.02155>.
- Rafailov, Rafael, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn (2023). “Direct Preference Optimization: Your Language Model Is Secretly a Reward Model”. In: *Advances in Neural Information Processing Systems*. Vol. 36. URL: <https://arxiv.org/abs/2305.18290>.
- Rajbhandari, Samyam, Jeff Rasley, Olatunji Ruwase, and Yuxiong He (2020). “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models”. In: *SC20: International Conference*

- for High Performance Computing, Networking, Storage and Analysis, pp. 1–16. DOI: [10.1109/SC41405.2020.00024](https://doi.org/10.1109/SC41405.2020.00024).
- Rebuffi, Sylvestre-Alvise, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert (2017). “iCaRL: Incremental Classifier and Representation Learning”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2001–2010. DOI: [10.1109/CVPR.2017.587](https://doi.org/10.1109/CVPR.2017.587).
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov (2017). *Proximal Policy Optimization Algorithms*. arXiv: [1707.06347](https://arxiv.org/abs/1707.06347). URL: <https://arxiv.org/abs/1707.06347>.
- Shin, Hanul, Jung Kwon Lee, Jaehong Kim, and Jiwon Kim (2017). “Continual Learning with Deep Generative Replay”. In: *Advances in Neural Information Processing Systems*. URL: <https://arxiv.org/abs/1705.08690>.
- Shoeybi, Mohammad, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro (2019). *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. arXiv: [1909.08053](https://arxiv.org/abs/1909.08053). URL: <https://arxiv.org/abs/1909.08053>.
- Shumailov, Ilia et al. (2024). “AI Models Collapse When Trained on Recursively Generated Data”. In: *Nature* 631, pp. 755–759. DOI: [10.1038/s41586-024-07566-y](https://doi.org/10.1038/s41586-024-07566-y).
- Sun, Yu, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt (2020). “Test-Time Training with Self-Supervision for Generalization under Distribution Shifts”. In: *Proceedings of the 37th International Conference on Machine Learning*, pp. 9229–9248. URL: <https://proceedings.mlr.press/v119/sun20b.html>.
- Wang, Dequan, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell (2021). “Tent: Fully Test-Time Adaptation by Entropy Minimization”. In: *International Conference on Learning Representations*. URL: <https://openreview.net/forum?id=uXl3bZLkr3c>.